sciendo

Proceedings on Privacy Enhancing Technologies ; 2021 (2):1–18

Adithya Vadapalli, Fattaneh Bayatbabolghani, and Ryan Henry*

You May Also Like... Privacy

Recommendation Systems Meet PIR

Abstract: We describe the design, analysis, implementation, and evaluation of PIRSONA, a digital content delivery system that realizes collaborative-filtering recommendations atop private information retrieval (PIR). This combination of seemingly antithetical primitives makes possible—for the first time—the construction of practically efficient e-commerce and digital media delivery systems that can provide personalized content recommendations based on their users' historical consumption patterns while simultaneously keeping said consumption patterns private. In designing PIRSONA, we have opted for the most performant primitives available (at the expense of rather strong non-collusion assumptions); namely, we use the recent computationally 1-private PIR protocol of Hafiz and Henry (PETS 2019.4) together with a carefully optimized 4PC Boolean matrix factorization.

Keywords: Multiparty computation; distributed point functions; private information retrieval; recommendation systems

DOI Editor to enter DOI
Received ..; revised ..; accepted ...

1 Introduction

Over the last two decades, we have witnessed a surge in electronic commerce and digital media streaming, with ubiquitous video streaming services like *Netflix*, *Hulu*, *YouTube*, and *Amazon Instant Video* overtaking cable and DVDs as the *de facto* source for television shows and films; audio streaming services like *Spotify*, *Apple Music*, *Last.fm*, and *Pandora* supplanting CDs and radio as the *de facto* choice for music and commentary; app stores like *Google Play*, Apple's *App Store*, the *Microsoft Store*, and the *Amazon Appstore* becoming the *de facto* way to download software; and e-book marketplaces like the *Amazon Kindle Store*, *Google eBooks*, and Barnes and Noble's *Nook Store* replacing brick-and-mortar bookstores as the *de facto* way to buy books.

Adithya Vadapalli: Indiana University, E-mail: avadapal@indiana.edu
***Corresponding Author: Ryan Henry:** University of Calgary, E-mail: ryan.henry@ucalgary.ca
Fattaneh Bayatbabolghani: Google, E-mail: fattaneh.bayat@gmail.com

With this shift to the digital, on-demand distribution of media comes increased convenience and cost savings for consumers—and an unprecedented capacity for content distributors to amass enormous quantities of fine-grain consumption-pattern data. And there's the rub: While content distributors can (and do) leverage the novel insights gleaned from such data to fuel recommendation engines and inform content creators in ways that benefit consumers, they also can (and do) sell or otherwise exploit said data for uses that are decidedly *not* in the consumers' best interests. Indeed, such data is invaluable to unscrupulous marketers and identity thieves seeking to defraud, law enforcement officers seeking to implicate or entrap, and hostile foreign agents seeking to manipulate and sow discord in democratic processes.

Given this tension between the utility of consumption-patterns on one hand and the sensitivity of consumption-patterns on the other, we ask the following question:

> *Is it possible to construct scalable, practically efficient digital content–distribution services that provide personalized content recommendations based on users' consumption patterns, all the while keeping those consumption patterns private?*

In this paper, we answer the preceding question in the affirmative by constructing PIRSONA, a novel system that uses (semi-honest) secure 4-party computation to implement collaborative-filtering recommendations atop a private information retrieval (PIR) protocol. This is no trivial task, as the privacy guarantees of PIR—allowing users to fetch items from a remote database while hiding from the data holder(s) which items they fetch—appear to be at fundamental odds with the goal of collaborative filtering—making automatic predictions about the interests and preferences of a user based on the interests and preferences of other like-minded users.

1.1 System overview

At a high level, our construction works as follows. Several *content distributors* hold identical copies of some database of fetchable records. To maintain their privacy, the users fetch records of interest using a fast, "computationally 1-private" multiserver PIR protocol (i.e., PIR that protects against a single malicious server). In addition to servicing the users' fetch requests, the servers store per-user, secret-shared consumption histories, which they extract directly from the users' incoming PIR queries. Periodically, the servers transform these consumption histories into (still secret-shared) collaborative filtering item and user profiles using a bespoke 4PC realiza-

BY-NC-ND

tion of Boolean matrix factorization. The latter secret-shared profiles are thenceforth used to render oblivious, yet personalized content recommendations for PIRSONA users.

1.2 Contributions

Our main contribution is the design, analysis, implementation, and evaluation of PIRSONA, a novel system that realizes collaborative filtering recommendations atop multiserver PIR. PIRSONA is the first system enabling untrusted content distributors to serve users obliviously while maintaining their ability to train recommender engines with which to provide oblivious, yet personalized content recommendations. Unlike in prior work on privacy-preserving machine learning [48, 49] or targeted advertising [37, 38, 56], users of PIRSONA need not maintain any local state, nor must they disclose any personal information to third parties or actively participate in the training of models; thus, PIRSONA delivers a user experience akin to now-ubiquitous non-private content distribution services—only with strong, provable privacy guarantees for its users. Toward realizing PIRSONA, we introduce some new primitives of independent interest, including (i) efficient 2- and 4PC fixed–selection wire multiplexers (MUXs) and demultiplexers (DeMUXs), (ii) fast 4PC vector normalization for secret-shared fixed-point vectors, (iii) 4PC well-formedness tests for $(2, 2)$-distributed point functions, and (iv) lightweight 3PC integer ("less than") comparison.

2 Preliminaries

We begin our technical discussion by introducing the basic technologies underlying our construction: Hafiz–Henry computationally 1-private PIR [31], Boolean matrix factorization–based collaborative filtering, and Du–Atallah 3-party multiplication [20]. Additionally, we provide a bird's-eye overview of how PIRSONA uses the latter secure MPC to "glue" the first two (seemingly antithetical) primitives into a seamless, privacy-preserving content distribution platform.

2.1 Private information retrieval

PIR is a cryptographic primitive using which users can fetch items from *remote* and *untrusted* database servers without disclosing to the servers which items they fetch. We focus on a specific class of PIR protocols—the so-called "1-private" multiserver scheme of Hafiz and Henry [31]—which provide extremely low download and server-side computation costs. These protocols provide the best-available mix of *efficiency* and *expressiveness* for the media delivery applications we target with PIRSONA. Here "expressiveness" refers to the ability of users to request content using human-memorable identifiers (i.e., titles or keywords) instead of physical addresses (i.e., row numbers) as required by most PIR. The remainder of this subsection briefly explains the Hafiz–Henry scheme.

Consider a database $\boldsymbol{D}$ modelled as an r-by-s matrix over a binary field $\mathbf{GF}(2^w)$. Here each of the r rows comprising $\boldsymbol{D}$ is a distinct *record* (e.g., a video, an e-book, or an app) composed of s-many w-bit *words*, for some suitably chosen bitlength w. A complete replica of $\boldsymbol{D}$ resides at each of $s+1$ *pairwise non-colluding* database servers (i.e., privacy requires that no two servers collude). We denote the servers by $\mathsf{P}_0, \ldots, \mathsf{P}_s$.[1]

2.1.1 Hafiz–Henry "perfectly 1-private" PIR

A user wishing to fetch $\vec{\boldsymbol{D}}_j$, the jth record in $\boldsymbol{D}$, constructs a *query template* by sampling a length-r vector $\vec{\boldsymbol{q}} \in_{\mathrm{R}} [0\,..\,s]^r$ uniformly at random subject to $\vec{\boldsymbol{q}}[j] = 0$. Here and throughout, $[0\,..\,s]$ refers to the set of integers between 0 and s (inclusive), while $\vec{\boldsymbol{q}}[j] \in [0\,..\,s]$ refers to the jth component of the vector $\vec{\boldsymbol{q}}$. Next, the user selects a permutation $\sigma\colon [0\,..\,s] \to [0\,..\,s]$ uniformly at random and then, for each $k \in [0\,..\,s]$, it computes and sends to P_k the vector

$$\mathsf{query}_k \coloneqq \vec{\boldsymbol{q}} + \sigma(k) \cdot \vec{\boldsymbol{e}}_j\,, \tag{1}$$

where $\vec{\boldsymbol{e}}_j \coloneqq \langle\, 0 \;\; 0 \cdots 1 \cdots 0 \,\rangle \in \mathbb{Z}^r$ denotes the jth length-r standard basis vector. The permutation σ forces independence between the jth component of query_k and the index k.

Upon receiving its vector query_k from the user, P_k computes and sends back to the user

$$\mathsf{response}_k \coloneqq \bigoplus\nolimits_{i=1}^{r} \vec{\boldsymbol{D}}_i(\mathsf{query}_k[i])\,, \tag{2}$$

a scalar in $\mathbf{GF}(2^w)$. As above, $\mathsf{query}_k[i]$ refers to the ith component of query_k, whereas $\vec{\boldsymbol{D}}_i(x) \coloneqq \vec{\boldsymbol{D}}_i[x]$, the xth word of $\vec{\boldsymbol{D}}_i$, unless $x = 0$, in which case $\vec{\boldsymbol{D}}_i(0) \coloneqq 0^w$.

Finally, for each $k \in [1\,..\,s]$, the user computes the $\sigma(k)$th word of $\vec{\boldsymbol{D}}_j$ from the pair $(\mathsf{response}_k, \mathsf{response}_0)$ using

$$\vec{\boldsymbol{D}}_j[\sigma(k)] = \mathsf{response}_k \oplus \mathsf{response}_0\,. \tag{3}$$

Hafiz–Henry perfectly 1-private PIR sharply meets concrete lower bounds for expected download [8, Thm 2.5] and server-side computation [5, Thm 6.4] costs; its client-side computation costs are likewise extremely low. This leaves only the upload cost as a target for optimization. The computationally 1-private variant reduces the per-server upload cost from $\Theta(r \lg s)$ to $\Theta\big((\lg r)(\lg s)\big)$, with no effect on the download cost and with only a nominal increase in the client- and server-side computation costs.

2.1.2 Hafiz–Henry "computationally 1-private" PIR

The computational variant uses *distributed point functions* [24] to compactly represent the vectors

1 One should think of the number of available servers ($s+1$) and record size as together dictating both the number of words per record (s) and the bit length of each word (w)—not the converse.

$\mathsf{query}_0, \ldots, \mathsf{query}_s$ in an otherwise-unmodified instance of the perfectly 1-private protocol. The scheme is most efficient (and easiest to state) when $s+1$ is a power of 2; we describe here—and assume throughout—this special case and refer to Hafiz and Henry's paper [31, §5.3] for details on the case where $s+1$ is not a power of 2.

Distributed point functions

Let $\mathbb{G}$ be an additive group and $\lambda \in \mathbb{N}$ a security parameter. Intuitively, a *2-out-of-2 distributed point function scheme* (*(2, 2)-DPF*) is an ordered pair (Gen, Eval) such that $(\mathsf{seed}_0, \mathsf{seed}_1) \leftarrow \mathsf{Gen}(1^\lambda, ; x, y)$ and Eval collectively define a $(2,2)$-additive sharing of the *point function* with domain $[1..r]$ and point $(x, y) \in [1..r] \times \mathbb{G}$. In particular, a triplet $(\mathsf{Eval}; \mathsf{seed}_0, \mathsf{seed}_1)$ so generated provides

1. **(perfect) correctness**: if $i \in [1..r]$, then $\mathsf{Eval}(\mathsf{seed}_0, i) - \mathsf{Eval}(\mathsf{seed}_1, i) = y$ if $i = x$ and 0 otherwise; and
2. **(computational) hiding**: given only one of seed_0 or seed_1, no PPT algorithm can distinguish a $(2,2)$-DPF with point (x, y) from a $(2,2)$-DPF with some other point $(x', y') \in [0..r] \times \mathbb{G}$, except with a probability negligible in λ.

Hafiz–Henry uses an efficient $(2,2)$-DPF of Boyle, Gilboa, and Ishai [10, Fig. 1] with 1-bit outputs, which is instantiable using any length-doubling pseudorandom generator (PRG). That construction benefits from the existence of a fast *full-domain evaluation* algorithm [10, §3.2.1], which efficiently expands seed_b into the length-r vector

$$\mathsf{EvalFull}(\mathsf{seed}_b) := \langle \mathsf{Eval}(\mathsf{seed}_b, 1), \ldots, \mathsf{Eval}(\mathsf{seed}_b, r) \rangle .$$

Construction for $s+1$ a power of 2

Suppose $s+1 = 2^L$ for some integer $L \geq 2$. A user seeking $\vec{D}_j$, the jth record in D, first samples an L-fold sequence of $(2,2)$-DPF seed pairs, $(\mathsf{seed}_0^{(L-1)}, \mathsf{seed}_1^{(L-1)}), \ldots, (\mathsf{seed}_0^{(0)}, \mathsf{seed}_1^{(0)})$, each with domain $[1..r]$ and point $(j, 1) \in [0..r] \times \mathbf{GF}(2)$. Next, for each $k \in [0..s]$, it sends the L-tuple

$$\mathbf{seed}_k := (\mathsf{seed}_{k_{L-1}}^{(L-1)}, \ldots, \mathsf{seed}_{k_1}^{(1)}, \mathsf{seed}_{k_0}^{(0)}) \tag{4}$$

to P_k. Here $(k_{L-1} \cdots k_1\ k_0)_2$ is the 0-padded L-bit binary representation of P_k's index k; i.e., $k = \sum_{i=0}^{L-1} k_i\, 2^i$.

Upon receiving its L-tuple $\mathbf{seed}_k$ from the user, P_k invokes EvalFull to expand each of the L seeds into a length-r vector of bits, and then it constructs query_k as the componentwise concatenation of the resulting bit vectors; that is, for each $i \in [1..r]$, it sets

$$\mathsf{query}_k[i] := \mathsf{Eval}(\mathsf{seed}_{k_{L-1}}^{(L-1)}, i) \| \cdots \| \mathsf{Eval}(\mathsf{seed}_{k_0}^{(0)}, i) . \tag{5}$$

From here, the protocol proceeds identically to its perfectly 1-private counterpart.

2.2 Collaborative filtering

Collaborative filtering is among the most widely deployed recommender engines. Broadly speaking, collaborative filtering renders predictions about the interests and preferences of a user using observations about the interests and preferences of many other users, based (very roughly) on the assumption that users who exhibit similar preferences about some things will likely exhibit similar preferences about other things. PIRSONA leverages a well-known *latent-factor collaborative filtering* algorithm based on Boolean matrix factorization [41]. Latent-factor collaborative filtering characterizes observed preferences with respect to some (parameterized) number of latent features; similarities with respect to these latent features form the basis for predictions about users' as-yet-unobserved preferences. Here the qualifier "latent" references the fact that the features are not necessarily tangible attributes like *artist* or *genre*.

Latent-factor collaborative filtering via matrix factorization

Consider a system in which there are m distinct users fetching items from a database comprising r distinct records and let $\boldsymbol{M} \in \{0,1\}^{m \times r}$ denote the $(0,1)$-matrix having one row per user and one column per record and having $\boldsymbol{M}_{ij} = 1$ if and only if the ith user has fetched the jth record.

Given $\boldsymbol{M}$, the goal of the recommender system is to recommend (i, j) pairs for which $\boldsymbol{M}_{ij} = 0$ and yet the ith user "should" enjoy the jth record. Toward this goal, consider a *user-profile* matrix $\boldsymbol{U} := [\vec{\boldsymbol{u}}_1; \cdots; \vec{\boldsymbol{u}}_m] \in \mathbb{R}^{m \times d}$ and *item-profile* matrix $\boldsymbol{V} := [\vec{\boldsymbol{v}}_1; \cdots; \vec{\boldsymbol{v}}_r] \in \mathbb{R}^{r \times d}$, respectively associating a profile from $(\mathbb{R} \cap [0,1])^d$ to each of the m users and to each of the r records reflected in $\boldsymbol{M}$. Here $d \in \mathbb{N}$ is a dimensionality parameter indicating the number of latent features needed to effectively capture preferences in the system. Initially, both $\boldsymbol{U}$ and $\boldsymbol{V}$ are selected uniformly subject to $\|\vec{\boldsymbol{u}}_i\|_2 = 1$ and $\|\vec{\boldsymbol{v}}_j\|_2 = 1$ for all $i \in [1..m]$ and $j \in [1..r]$; the goal is then to "evolve" $\boldsymbol{U}$ and $\boldsymbol{V}$ until $\boldsymbol{U}\boldsymbol{V}^\intercal \in (\mathbb{R} \cap [0,1])^{m \times r}$ is a "reasonable approximation" to $\boldsymbol{M}$, and then to use the differences between $\boldsymbol{U}\boldsymbol{V}^\intercal$ and $\boldsymbol{M}$ as the basis for making recommendations.

To this end, we seek an approximate solution to the minimization problem

$$\underset{\boldsymbol{U},\boldsymbol{V}}{\arg\min} \Big\{ \tfrac{1}{\|\boldsymbol{M}\|_1} \sum_{\boldsymbol{M}_{ij}=1} \big(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle\big)^2 + \mu_u \sum_{i=1}^{m} \|\vec{\boldsymbol{u}}_i\|_2^2 + \mu_v \sum_{j=1}^{r} \|\vec{\boldsymbol{v}}_j\|_2^2 \Big\},$$

for some convergence parameters $\mu_u \in \mathbb{R}^+$ and $\mu_v \in \mathbb{R}^+$.

There are several known methods for solving such minimization problems. Perhaps the best-known method—and the method employed by PIRSONA—is *gradient descent*, a first-order iterative algorithm for approximating (local) minima of any differentiable function. Each gradient-descent iteration produces progressively "better" approximations (by normalizing the vectors obtained) via the *adaptation rules*

$$\vec{\boldsymbol{u}}_i \leftarrow \hat{\mu}_u \vec{\boldsymbol{u}}_i + \gamma \sum_{\boldsymbol{M}_{ij}=1} \vec{\boldsymbol{v}}_j \big(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle\big) \tag{6}$$

and

$$\vec{\boldsymbol{v}}_j \leftarrow \hat{\mu}_v \vec{\boldsymbol{v}}_j + \gamma \sum_{\boldsymbol{M}_{ij}=1} \vec{\boldsymbol{u}}_i \big(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle\big), \tag{7}$$

for $\hat{\mu}_u = 2\mu_u - 1$, $\hat{\mu}_v = 2\mu_v - 1$, and step size $\gamma \in \mathbb{R}^+$.

After several gradient-descent iterations, the resulting $\boldsymbol{U}$ and $\boldsymbol{V}$ are used to render recommendations for the ith user as

follows: Compute the length-r vector $\vec{\boldsymbol{R}}_i := \vec{\boldsymbol{u}}_i \boldsymbol{V}^\intercal - \vec{\boldsymbol{M}}_i$, where $\vec{\boldsymbol{M}}_i$ denotes the ith row of $\boldsymbol{M}$; the largest components of $\vec{\boldsymbol{R}}_i$ are the "top" recommendations for the ith user.

2.3 Du–Atallah 3-party multiplication

In principle, any secure MPC protocol could serve as the substrate through which PIRSONA performs collaborative filtering over PIR queries. However, as our design already uses 1-private PIR among 2^L pairwise non-colluding servers, we focus our attention on 4PC protocols secure when all parties are semi-honest. This special subclass of MPC admits computations that are highly efficient relative to, e.g., 2PC from garbled circuits or general MPC secure against threshold collusion or minorities of Byzantine computation parties.

We employ Du–Atallah multiplication [20], an efficient 1-private 3-party variation of Beaver triples [4]. Suppose P_0 and P_1 respectively hold $(2,2)$-additive shares (x_0, y_0) and (x_1, y_1) of a secret pair (x, y) from some ring R; the goal is for them to compute shares z_0 and z_1 of the product $xy \in R$. A semi-trusted third party, P_2, facilitates this computation. At a high level, the idea is for P_0 and P_1 to exchange blinded copies of their respective shares so that both parties effectively hold (x_0, y_0, x_1, y_1), albeit with additional machinery to maintain secrecy. P_2 chooses *blinding factors* and *cancellation terms* to help P_0 and P_1 arrive at the final product; that is, P_2 samples $X_0, X_1, Y_0, Y_1, \alpha \in_{\mathrm{R}} R$, and then it sends $(X_0, Y_0, X_0 Y_1 + \alpha)$ to P_0 and $(X_1, Y_1, X_1 Y_0 - \alpha)$ to P_1.

P_1 then sends $(x_1 + X_1, y_1 + Y_1)$ to P_0, and P_0 outputs

$$z_0 := x_0\big(y_0 + (y_1 + Y_1)\big) - Y_0(x_1 + X_1) + (X_0 Y_1 + \alpha), \tag{8}$$

while P_0 sends $(x_0 + X_0, y_0 + Y_0)$ to P_1, and P_1 outputs

$$z_1 := x_1\big(y_1 + (y_0 + Y_0)\big) - Y_1(x_0 + X_0) + (X_1 Y_0 - \alpha). \tag{9}$$

It is easy to verify that Du–Atallah multiplication is correct—i.e., that $z_0 + z_1 = (x_0 + x_1)(y_0 + y_1) = xy$—and that the view of each party to the computation is (perfectly) simulatable. Also notice that all contributions of P_2 can be computed offline before P_0 and P_1 ever receive their private shares. Finally, we remark that Du–Atallah multiplication generalizes naturally to inner products [19] and efficient scalar-vector multiplication.

2.4 Secure 4PC as "garbled glue"

Whenever the ith user submits a PIR query over the database, some designated subset of the servers stores a tuple (i, seed_b), where seed_b is one of the L seeds constituting that query. Periodically, the designated servers perform a 4-party computation that takes as input (i) these (i, seed_b) pairs and (ii) the previously computed user- and item-profile shares, and outputs updated user- and item-profile shares. The updated user- and item-profile shares can be multiplied to produce a personalized content recommendation vector upon which to base user-specific recommendations.

3 Threat model

As PIRSONA composes *computationally 1-private* PIR with MPC secure against a *single passive corruption*, three assumptions are immediate and unavoidable; namely, that

1. the adversary is computationally bounded,
2. the adversary is honest-but-curious, and
3. the adversary controls *at most one* server (out of 2^L).

PIRSONA's reliance on computational assumptions follows from its use of $(2,2)$-DPFs, which are instantiable from any length-doubling PRG; thus, beyond the (admittedly quite strong) assumption of non-collusion, PIRSONA relies only on rather conservative "minicrypt" assumptions [36]. For instance, our implementation assumes only that AES-128 is hard to distinguish from an ideal random permutation.

PIRSONA employs a novel 4PC sanity-check protocol for the DPFs to ensure that users cannot corrupt the user- and item-profile shares by submitting malformed queries; thus, we can safely assume that the adversary controls—in addition to one server—an arbitrary number of Byzantine users. This is an important aspect of the model, as contributing strategically chosen queries may enable the adversary to influence the collaborative filtering profiles in a way that causes the model to leak information about the consumption histories of honest users [11]. We stress that PIRSONA does not attempt to limit such collaborative filtering model leakage (though doing so would be an interesting direction for future work); rather, we merely insist that the only source of information leakage is that which is inherent to the collaborative filtering recommendations.

Finally, we assume that all user–server and server–server communication occurs over point-to-point confidential and mutually authenticated channels. We do not concern ourselves with how these channels are implemented (presumably TLS coupled with password-based authentication).

4 Overview of the 4PC for gradient descent

We now describe, at a high level, the 4PC protocol PIRSONA uses to evaluate Equations (6) and (7), the so-called adaptation rules for gradient descent–based Boolean matrix factorization. Given an incoming query from user i for item j, a designated subset of servers must cooperatively

1. locate the user profile $\vec{\boldsymbol{u}}_i$ and item profile $\vec{\boldsymbol{v}}_j$;

2. compute $\Delta_{\boldsymbol{u}_i} := \vec{\boldsymbol{v}}_j(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle)$ and $\Delta_{\boldsymbol{v}_j} := \vec{\boldsymbol{u}}_i(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle)$;
3. update $\vec{\boldsymbol{u}}_i$ with $\Delta_{\boldsymbol{v}_j}$ and $\vec{\boldsymbol{v}}_j$ with $\Delta_{\boldsymbol{u}_i}$; and, ultimately,
4. normalize the resulting updated profiles.

In the following, we elaborate on each step in turn.

Step 1a: Locating the user profile. This step is made trivial by virtue of all user–server communication occurring over mutually authenticated channels; that is, because each server knows with which user it is communicating, it can easily retrieve the correct entry from its list of user-profile shares.

Step 1b: Locating the item profile. Recall that users fetch items via Hafiz–Henry computationally 1-private PIR. In that protocol, the user submits an L-fold sequence $(\mathsf{seed}^{(L-1)}_{k_{L-1}}, \ldots, \mathsf{seed}^{(1)}_{k_1}, \mathsf{seed}^{(0)}_{k_0})$ of DPF seeds to each server (see §2.1.2). Assume, without loss of generality, that servers P_0 and P_2 receive $\mathsf{seed}^{(0)}_0$ while P_1 and P_3 receive $\mathsf{seed}^{(0)}_1$. The four parties use their respective seeds to fetch shares of $\vec{\boldsymbol{v}}_j$ using the 4-party MUX construction detailed in Section 6.2.2. For technical reasons (explained in Section 6.2), the 4-party MUX ultimately results in P_0 and P_1 respectively holding shares (v_0, b_0) and (v_1, b_1) of $((-1)^b \vec{\boldsymbol{v}}_j, (-1)^b)$, where b is a uniformly distributed sign bit.

Step 2: Computing $\Delta_{\boldsymbol{u}_i}$ and $\Delta_{\boldsymbol{v}_j}$. The computation of $\Delta_{\boldsymbol{u}_i}$ and $\Delta_{\boldsymbol{v}_j}$ proceeds via a 4-move 3PC protocol comprising three sequential rounds of generalized Du–Atallah multiplications (see §2.3). Note that after Step 1, P_0 and P_1 respectively hold shares (u_0, v_0, b_0) and (u_1, v_1, b_1) of $(\vec{\boldsymbol{u}}_i, (-1)^b \vec{\boldsymbol{v}}_j, (-1)^b)$. P_2 makes the first move, sending a sequence of Du–Atallah blinding factors and cancellation terms for P_0 and P_1 to use in the subsequent moves. In the second move, P_0 and P_1 use (u_0, v_0, b_0) and (u_1, v_1, b_1) to compute shares z_0 and z_1 of $(-1)^b(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle)$. In the third move, P_0 and P_1 use (b_0, z_0) and (b_1, z_1) to compute shares z'_0 and z'_1 of $(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle)$. In the fourth and final move, P_0 and P_1 use (v_0, z_0) and (v_1, z_1) to compute shares of $\Delta_{\boldsymbol{u}_i} = \vec{\boldsymbol{v}}_j(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle)$, and they use (u_0, z'_0) and (u_1, z'_1) to compute shares of $\Delta_{\boldsymbol{v}_j} = \vec{\boldsymbol{u}}_i(1 - \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{v}}_j \rangle)$.

Step 3a: Updating $\vec{\boldsymbol{u}}_i$ with $\Delta_{\boldsymbol{v}_j}$. Analogous to Step 1a, this step is made trivial by virtue of all user–server communication occurring over mutually authenticated channels.

Step 3b: Updating $\vec{\boldsymbol{v}}_j$ with $\Delta_{\boldsymbol{u}_i}$. This step is roughly converse to Step 2b; thus, whereas Step 2b used seeds from the user's query to instantiate a 4-party MUX, this step uses them to instantiate a 4-party DeMUX, as detailed in Section 6.3.

Step 4: Normalizing the updated profiles. At this point, P_0 and P_1 hold shares $(u_0, \boldsymbol{V}_0)$ and $(u_1, \boldsymbol{V}_1)$ of the updated (but non-normalized) user profile $\vec{\boldsymbol{u}}_i$ and item-profile matrix $\boldsymbol{V}$; their goal is to normalize each of the shared profiles. We describe here only the normalization of $\vec{\boldsymbol{u}}_i$; normalization of the profiles comprising $\boldsymbol{V}$ is similar. The idea is for P_0 and P_1 (with help from P_2) to use Du–Atallah to compute shares U_0 and U_1 of $\|\vec{\boldsymbol{u}}_i\|_2^2 = \langle \vec{\boldsymbol{u}}_i, \vec{\boldsymbol{u}}_i \rangle$, and then to enlist the help of P_2 and P_3 to convert these into (shares of) a high-fidelity linear approximation to $1/\|\vec{\boldsymbol{u}}_i\|_2$. At a high level, the four parties do this by running a modified PIR protocol to jointly fetch shares of the coefficients for the linear approximation.

Section 6 presents complete details and analyses of the 4-party MUX used in Step 1b (§6.2), the 4-party DeMUX used in Step 3b (§6.3), and the 4-party vector-normalization protocol used in Step 4 (§6.4). Following that, Section 7 describes one potential way to *use* the resulting item and user profiles—an efficient 3PC that takes as input (i) a uniformly sampled number $p \in ([0, 1] \cap \mathbb{R})$ and (ii) shares of the resulting user profile $\vec{\boldsymbol{u}}_i$ and item-profile matrix $\boldsymbol{V}$, and outputs an obliviously sampled, personalized recommendation for the ith user.

5 System architecture

Before presenting the details of our MPC, we elaborate on the high-level PIRSONA architecture sketched out in the preceding sections. Recall that the system comprises 2^L pairwise non-colluding servers, $\mathsf{P}_0, \ldots, \mathsf{P}_{2^L}$, for some $L \geq 2$; of these, only four are involved in collaborative filtering profile updation. Specifically, P_0 and P_1 play a central role in all aspects of collaborative filtering, while P_2 participates in all 3PC sub-protocols and P_3 joins only for the occasional 4PC sub-protocol. Any additional servers merely respond to users' PIR queries.[2] In particular, P_0 and P_1 hold $(2, 2)$-additive shares $(\boldsymbol{U}_0, \boldsymbol{V}_0)$ and $(\boldsymbol{U}_1, \boldsymbol{V}_1)$ of the user- and item-profile matrices; P_2 and P_3 hold $\boldsymbol{V}_1$ and $\boldsymbol{V}_0$ in common with P_0 and P_1, respectively. Upon receiving a query from user i, P_0 and P_2 store the pair $(i, \mathsf{seed}^{(0)}_0)$ consisting of the user's identity and the "least-significant–bit" DPF seed from her query (see Equation (4)); likewise, P_1 and P_3 store the pair $(i, \mathsf{seed}^{(0)}_1)$. Periodically, the parties input these values to a 4PC to update the collaborative filtering profiles (see §4).

5.1 Query representation

By default, Hafiz–Henry assumes users fetch records via *positional queries* [30]; that is, via their row numbers within $\boldsymbol{D}$. We remark that (i) the DPFs naturally support keyword-based queries [14] as a more user-friendly alternative and (ii) this extension "plays well" with Hafiz–Henry queries. Beyond the preceding remark, we do not concern ourselves with how users specify which records they seek; however, we do prescribe one minor change to Hafiz–Henry query generation in support of PIRSONA functionality: The DeMUX assumes a so-called *leafless seed* for the "least-significant–bit" DPF (see §6.3) and, consequently, that the servers substitute

2 Hafiz–Henry grows increasingly efficient as the number of servers becomes large; this is why we entertain the possibility that $L > 2$.

the bit vector $\mathsf{flags}(\mathsf{seed}_b^{(0)})$, as defined in §6.1, in lieu of $\mathsf{EvalFull}(\mathsf{seed}_b^{(0)})$ when constructing query_k.

5.2 Profile representation

Recall that collaborative filtering user and item profiles are length-d unit vectors of positive reals (see §2.2), whereas Du–Atallah multiplication works over an arbitrary finite ring R (see §2.3). For concreteness—and to facilitate efficient implementation on 64-bit computers—PIRSONA fixes R as the ring of integers modulo 2^{64}; profiles are then vectors of 64-bit unsigned integers representing fixed-point approximations to reals. Specifically, the fixed-point approximation to x is $\lfloor x\, 2^p \rfloor$, where $p \in \mathbb{N}$ is a *fractional precision* parameter indicating how many bits are used to represent the fractional part of x. Since profile components are non-negative and bounded above by 1, we require high-fidelity fractional approximations. Yet setting p too large can cause intermediate values to overflow, as the product of fixed-point numbers with fractional precisions p_1 and p_2 has fractional precision $p_1 + p_2$ (and summing n such numbers grows the integer part by up to $\lg n$ bits).

When $\vec{u}_i$ and $\vec{v}_j$ are represented by vectors of fixed-point numbers with fractional precision p, the gradient-descent adaptation rules yield vectors $\Delta_{u_i} = \vec{v}_j(1 - \langle \vec{u}_i, \vec{v}_j \rangle)$ and $\Delta_{v_j} = \vec{u}_i(1 - \langle \vec{u}_i, \vec{v}_j \rangle)$ with fractional precision $3p$. For technical reasons (see §6.4.1), we must ensure that the most-significant bit is unset in every vector component; hence, we set $p = 64 - 3p - N - 1$ with $N := \lceil (\lg(n+1))/(1+\epsilon) \rceil$, where n is a tuning parameter determining the number of adaptation values the servers can accumulate without risk of overflows and $\epsilon \geq 0$ captures the worst-case approximation error in our 4-party normalization protocol (see §6.4).

We emphasize that our MPC protocols are agnostic to the fixed-point representation up until the *normalization* step (see §6.4), which includes *precision-reduction* substeps (§6.4.1) to "reset" the fractional precision as needed.

6 Primitives for gradient descent

We begin with an overview of the Boyle–Gilboa–Ishai (2, 2)-DPF construction [10, Fig. 1], which serves as the scaffolding around which we construct our 4PC gradient descent.

6.1 Boyle–Gilboa–Ishai (2,2)-DPFs

The presentation herein attempts to strike a balance between focusing on the intuition behind the Boyle–Gilboa–Ishai construction and clearly articulating specific low-level details upon which our MUXs and DeMUXs rely.

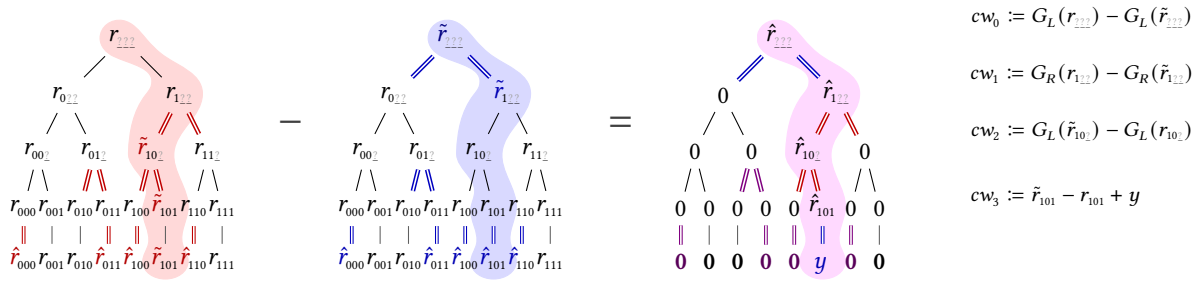
The Goldreich–Goldwasser–Micali PRF

At its core, Boyle et al.'s construction is a clever generalization of the celebrated Goldreich–Goldwasser–Micali (GGM) construction [25] of a *pseudorandom function (PRF)* from any length-doubling PRG. The GGM construction represents a PRF taking h-bit inputs by a height-h binary tree in which (i) the root node is labelled by the λ-bit PRF key K (i.e., a PRG seed); and (ii) the left and right children of each non-leaf node are respectively labelled by the left and right halves of the image of the PRG applied to the label on the parent. Notice that there is a natural bijection between the set of h-bit binary strings and the leaves of a tree so defined, with the sequence of left (meaning the next bit is 0) or right (meaning the next bit is 1) traversals from the root to any leaf associating that leaf with a specific h-bit input string. GGM defines the image $F_K(i)$ of the PRF keyed by $K \in \{0,1\}^\lambda$ and evaluated at $i \in \{0,1\}^h$ as the *label* on the leaf node associated with i when this natural bijection is applied to the height-h tree with root labelled K.

(2,2)-DPFs from pairs of GGM-like trees

Boyle et al.'s construction maintains the basic structure of GGM, while adding some additional machinery in the form of so-called *correction words (CWs)*; that is, traversing the "DPF tree share" for a given input follows the same basic process as in GGM: Begin at the root and repeatedly traverse either left or right, depending on successive bits of the input, until arriving at a leaf node. The main difference is in how DPF-tree traversal incorporates the CWs into the computation of labels. At a high level, the strategy is to single out the path from the root to a specific *target leaf node* (the x-coordinate of the point function's "point"), and then to specially craft the CWs to force the labels on all *other* paths in a given pair of GGM-like trees to converge, thereby assigning identical labels to all non-target leaves. To this end, the construction associates with each layer of the tree shares one λ-bit CW, the two least-significant bits of which are special *toggle bits* that help the evaluator decide to which nodes it should "apply" the CW. The precise details of how to generate and apply CWs (and toggle bits) goes beyond what is needed to understand our MUX and DeMUX constructions; for brevity, we omit those details here and direct interested readers to Gilboa and Ishai [24] and Boyle et al. [10]. For our purposes, it suffices to know that

1. each node (in each tree share) is associated with its own *flag bit* that is determined by a combination of the toggle bits and node labels produced by the PRG output together with the CW for the preceding layer;
2. the distribution of flag bits within each tree share is *pseudorandom* (and *independent of the target leaf* node); and



(a) DPF tree share induced by seed_0 **(b)** DPF tree share induced by seed_1 **(c)** Nodewise difference of tree shares **(d)** Correction words

Fig. 1. The DPF trees (plus associated CWs) representing a $(2, 2)$-DPF with point $(5, y) \in [0..7] \times \mathbb{G}$. Each node connects with its children using single edges (i.e., '|') if its flag bit is 0 and using double edges (i.e., '‖') if its flag bit is 1. In the CW formulae in Figure 1d, $G_L(\cdot)$ and $G_R(\cdot)$ respectively denote the left and right halves of the image of some length-doubling pseudorandom generator G. The two seeds defining the DPF are $\text{seed}_0 := (r_{\underline{222}}; cw_0 \| \cdots \| cw_3)$ and $\text{seed}_1 := (\tilde{r}_{\underline{222}}; cw_0 \| \cdots \| cw_3)$.

3. the flag bits in corresponding nodes of the two DPF tree shares are *pairwise identical*—except along the path to the target leaf, where they are *pairwise complementary*.

In the following, we denote by $\text{flags}(\text{seed}_0)$ and $\text{flags}(\text{seed}_1)$ the vectors of leaf-layer flag bits for the trees respectively induced by the DPF seeds seed_0 and seed_1 for $(\text{seed}_0, \text{seed}_1) \leftarrow \text{Gen}(1^\lambda, r; x, y)$.

Taken together, the preceding facts yield the next observation, upon which we base our MUX constructions.

Observation 1. *If* $(\text{seed}_0, \text{seed}_1) \leftarrow \text{Gen}(1^\lambda, r; x, y)$*, then the vectors* $\text{flags}(\text{seed}_0)$ *and* $\text{flags}(\text{seed}_1)$ *define a computationally private* $(2, 2)$*-additive sharing of the* x*th standard basis vector of length* r *over* $\mathbf{GF}(2)$*; i.e.,* $\text{flags}(\text{seed}_0) \oplus \text{flags}(\text{seed}_1) = \vec{e}_x$.

Figure 1 illustrates the DPF trees (and associated CWs) for a $(2, 2)$-DPF with height $h = 3$ and point $(5, y)$. The leftmost two trees respectively depict the tree shares induced by seed_0 (Figure 1a) and seed_1 (Figure 1b); the rightmost tree (Figure 1c) denotes their nodewise difference.

6.2 DPFs as fixed–selection wire MUXs

An r-to-1 *multiplexer (MUX)* is a combinational logic circuit that, based on the signal it receives on a *selection wire*, forwards the value from one *input wire* (out of r) to a single *output wire*. That 2-party MUXs are instantiable from $(2, 2)$-DPFs is hardly surprising: Privately selecting one-out-of-r input wires is substantially equivalent to 2-server PIR over r items—the veritable "bread and butter" of $(2, 2)$-DPFs.

The above-noted parallel between MUXs and PIR notwithstanding, two important caveats differentiate DPFs in our MPC setting from their use in the more familiar PIR setting; specifically, existing DPF-based PIR protocols

1. operate on the leaf-node *labels*—indeed, the ubiquitous early-termination optimization [10, §3.2.3] treats each bit of each leaf-node label as a distinct output, effectively lopping $\lfloor \lg \lambda \rfloor$ layers off the DPF trees; and they
2. perform arithmetic in a binary field, using the DPF outputs only to decide which records to exclusive-OR.

Notably, working in the characteristic-2 setting of binary fields eliminates the notion of positive and negative *signs*. Even in settings where the characteristic is greater than 2, the querier can typically (i) flip the sign at reconstruction time or (ii) induce the desired sign via the leaf-layer CW.

In our setting, the MUX outputs $(2, 2)$-additive shares in $\mathbb{Z}_{2^{64}}$, which are input to some larger 4PC protocol; thus, *signs matter*. Yet, jumping ahead, we reserve the leaf-layer labels for use in demultiplexing (see §6.3) and use only the leaf-layer flag bits for multiplexing. (The alternative would be two DPFs—one for multiplexing and one for demultiplexing—plus a consistency check to ensure they share a target; we opt to use one DPF for efficiency: It halves storage cost and calls to EvalFull while obviating the need for an additional consistency check.) Using the flag bits precludes inducing the desired sign via the leaf-layer CW; instead, our MUX outputs (shares of) the sign bit so that subsequent MPC can flip the sign as needed.

6.2.1 2-party MUX with public input wires

The 2-party MUX is fast and simple, but it requires cleartext values on all input wires;[3] the 4-party MUX adds support for secret-shared input wires.

Suppose P_0 and P_1 respectively hold seed_0 and seed_1, while both parties hold $\boldsymbol{V} = [\vec{v}_1; \cdots; \vec{v}_r]$; the goal is for P_0 and P_1 to respectively output (v_0, b_0) and (v_1, b_1) such that $v_0 + v_1 = (b_0 + b_1)\vec{v}_j$. To this end, P_0 computes and outputs

$$(v_0, b_0) := \left(\textstyle\sum_{\text{flags}(\text{seed}_0)[j]=1} \vec{v}_j,\ \|\text{flags}(\text{seed}_0)\|_1\right) \tag{10}$$

while P_1 computes and outputs

$$(v_1, b_1) := \left(-\textstyle\sum_{\text{flags}(\text{seed}_1)[j]=1} \vec{v}_j,\ -\|\text{flags}(\text{seed}_1)\|_1\right). \tag{11}$$

Correctness of the above 2-party MUX construction follows immediately from Observation 1. We write $(v_B, b_B) \leftarrow \text{2PC-MUX}_B(\text{seed}_B, \boldsymbol{V})$ to indicate that (v_B, b_B) is output by P_B using Equation (10) if $B = 0$ or Equation (11) if $B = 1$.

3 For PIRSONA, this means P_0 and P_1 would hold cleartext item profiles, which would leak information beyond what is inherently leaked by collaborative filtering.

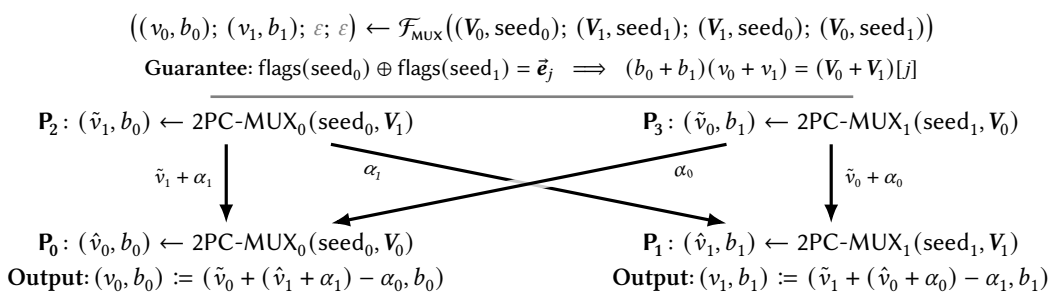


Fig. 2. 4-party fixed–selection wire MUX with secret-shared input wires, as described in Section 6.2.2.

6.2.2 4-party MUX with secret-shared input wires

The 4-party MUX is a 2-move protocol that supports $(2, 2)$-additively shared input wires. It follows a simple *mix-and-match* pattern that we will see again in the context of DPF sanity checks for our 4-party DeMUX (§6.3) and 4-party profile normalization (§6.4).

Let $\boldsymbol{V}_0$ and $\boldsymbol{V}_1$ be $(2, 2)$-additive shares of $\boldsymbol{V} = \big[\vec{\boldsymbol{v}}_1; \cdots; \vec{\boldsymbol{v}}_r\big]$. Suppose P_0 and P_2 both hold seed_0, while P_1 and P_3 both hold seed_1; likewise, suppose that P_0 and P_3 both hold $\boldsymbol{V}_0$, while P_1 and P_2 both hold $\boldsymbol{V}_1$. As in the 2-party MUX, the goal is for P_0 and P_1 to respectively output (v_0, b_0) and (v_1, b_1) such that $v_0 + v_1 = (b_0 + b_1)\vec{\boldsymbol{v}}_j$. To this end, in the first move:

1a) P_2 selects an output-wire blinding factor α_1, computes $(\hat{v}_1, b_0) \leftarrow \mathsf{2PC\text{-}MUX}_0(\mathsf{seed}_0, \boldsymbol{V}_1)$, and then sends α_1 to P_1 and $\hat{v}_1 + \alpha_1$ to P_0; meanwhile,

1b) P_3 selects an output-wire blinding factor α_0, computes $(\hat{v}_0, b_1) \leftarrow \mathsf{2PC\text{-}MUX}_1(\mathsf{seed}_1, \boldsymbol{V}_0)$, and then sends α_0 to P_0 and $\hat{v}_0 + \alpha_0$ to P_1.

For the second move:

2a) P_0 computes $(\tilde{v}_0, b_0) \leftarrow \mathsf{2PC\text{-}MUX}_0(\mathsf{seed}_0, \boldsymbol{V}_0)$ and $v_0 := \tilde{v}_0 + (\hat{v}_1 + \alpha_1) - \alpha_0$, and then it outputs (v_0, b_0); meanwhile,

2b) P_1 computes $(\tilde{v}_1, b_1) \leftarrow \mathsf{2PC\text{-}MUX}_1(\mathsf{seed}_1, \boldsymbol{V}_1)$ and $v_1 := \tilde{v}_1 + (\hat{v}_0 + \alpha_0) - \alpha_1$, and then it outputs (v_1, b_1).

An easy derivation confirms that $v_0 + v_1 = (-1)^b \vec{\boldsymbol{v}}_j$ with $b_0 + b_1 = (-1)^b$. Figure 2 illustrates the above 4-party MUX construction and its "mix-and-match" structure.

6.3 DPFs as fixed–selection wire DeMUXs

A 1-to-r *demultiplexer (DeMUX)* is a logic circuit that implements the converse of an r-to-1 MUX; that is, based on the signal it receives on a *selection wire*, it forwards the value from a single *input wire* to one *output wire* (out of r). Oblivious demultiplexing bears a striking resemblance to so-called PIR-Writing [52], a task for which DPFs have been fruitfully deployed in the contexts of, e.g., oblivious RAM [18, 42] and anonymous messaging [16].

Yet one crucial aspect of our DeMUXs differentiates them from standard PIR-Writing: in PIR-Writing, the user fixes values on *both* the selection wire *and* the input wire, whereas our DeMUX effectively *unfixes* the input wire, enabling the *servers*—as opposed to the user—to forward any (secret-shared) value of their choosing to an output wire determined by the (still fixed by the user) selection wire. All other output wires receive $(2, 2)$-additive shares of zero. We remark that Kushilevitz and Mour propose a superficially similar DPF-based 4-party PIR-Writing construction in the context of oblivious RAM [42, Figure 1]; however, unlike with our DeMUXs, the *user* in Kushilevitz-Mour must choose the value for the input wire (and it must do so at DPF generation time). This distinction makes our DeMUXs significantly more powerful and is essential for PIRSONA functionality, where the DPF generator (i.e., a PIRSONA user) is not privy to the value (i.e., a scalar multiple of $\Delta_{\boldsymbol{v}_j}$) to appear on the input wire.

Leafless DPF seeds

The "input-wire" of a Boyle–Gilboa–Ishai DPF is determined by the leaf-layer CW; thus, we "unfix" the input-wire value by deferring selection of this CW. To this end, we provide a modified Gen that emits $(2, 2)$-additive shares of a leaf-layer CW for $(j, 0) \in [1..r] \times \mathbb{G}$. With the latter shares, P_0 and P_1 can compute a leaf-layer CW corresponding to any $(2, 2)$-additively shared input wire value from $\mathbb{G}$.[4]

While DPF seeds without a leaf-layer CW are effectively *inevaluable*, the missing CW presents no obstacle to generating the interior of the DPF tree, up to and including the leaf-layer flag bits—it is only the (labels on) the leaf nodes that are not yet determined. Thus, such "defoliated" DPFs remain suitable for use in our 2- or 4-party MUXs (see §6.2). In a nod to arboriculture, we call the eventual computation of the leaf-layer CW for such leafless seeds *vernalization*. The modified seed-generation algorithm, $\mathsf{G\overline{en}}$, uses the default Gen as a black box; specifically, on input $(1^\lambda, ; j)$, it

1. samples $(\mathsf{seed}_0, \mathsf{seed}_1) \leftarrow \mathsf{Gen}(1^\lambda, ; j, 0)$;
2. parses each seed_b as $(K_b;\ cw_0 \| \cdots \| cw_{h-1} \| cw_h)$, with cw_h being the leaf-layer CW; and then
3. outputs $(\delta, cw_h - \delta)$ and $\overline{\mathsf{seed}}_0 := (K_0;\ cw_0 \| \cdots \| cw_{h-1})$ and $\overline{\mathsf{seed}}_1 := (K_1;\ cw_0 \| \cdots \| cw_{h-1})$

4 Note that only one leaf-layer CW can be computed per DPF seed pair, as two such CWs suffice to deduce the selection- and input-wire values.

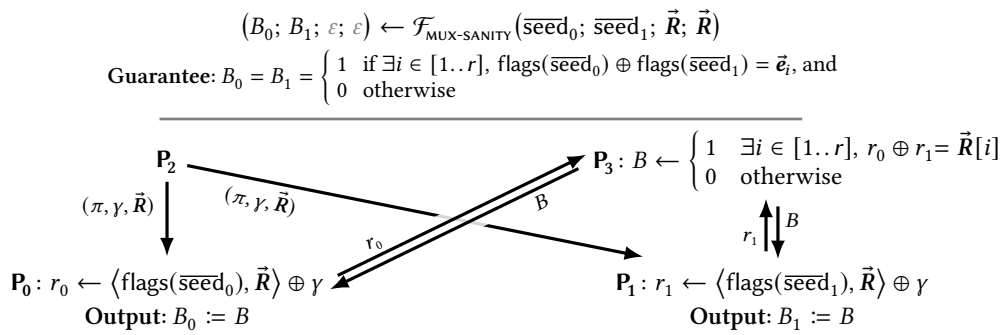


Fig. 3. The first component of the 4-party DPF sanity check protocol, as described in Section 6.3.2.1.

Vernalization is a 2-party interaction comprising a single move. Let y_0 and y_1 be $(2, 2)$-additive shares of a secret y and suppose P_0 and P_1 respectively hold $(\overline{\text{seed}}_0, \delta, y_0)$ and $(\overline{\text{seed}}_1, cw_h - \delta, y_1)$. P_0 computes $z_0 := y_0 + \delta$ and sends it to P_1; meanwhile, P_1 computes $z_1 := y_1 + (cw_h - \delta)$ and sends it to P_0. Both parties output the leaf-layer CW

$$z_0 + z_1 = (y_0 + \delta) + (y_1 + (cw_h - \delta)) = y + cw_h.$$

6.3.1 2-party DeMUX construction

Suppose P_0 and P_1 respectively hold $(\overline{\text{seed}}_0, \delta, y_0)$ and $(\overline{\text{seed}}_1, cw_h - \delta, y_1)$ such that $\text{flags}(\overline{\text{seed}}_0) \oplus \text{flags}(\overline{\text{seed}}_1) = \vec{e}_j$ and $\text{EvalFull}(\overline{\text{seed}}_0 \| cw_h) - \text{EvalFull}(\overline{\text{seed}}_1 \| cw_h) = \vec{0}$; the goal is for P_0 and P_1 to respectively output vectors V_0 and V_1 such that $V_0 + V_1 = y \cdot \vec{e}_j$. To this end, (i) P_0 and P_1 use 2-party vernalization to compute $cw'_h := cw_h + y$, and then (ii) they respectively output $V_0 := \text{EvalFull}(\overline{\text{seed}}_0 \| cw'_h)$ and $V_1 := \text{EvalFull}(\overline{\text{seed}}_1 \| cw'_h)$.

6.3.2 4-party DeMUX w/ sanity checks

The 2-party DeMUX assumes that the seeds held by P_0 and P_1 constitute a well-formed leafless DPF; that is, it assumes that, $\text{flags}(\overline{\text{seed}}_0) \oplus \text{flags}(\overline{\text{seed}}_1) = \vec{e}_j$ and $\text{EvalFull}(\overline{\text{seed}}_0 \| cw_h) - \text{EvalFull}(\overline{\text{seed}}_1 \| cw_h) = \vec{0}$. If not, the resulting output wires may take on arbitrary values, corrupting the entire item-profile matrix. The 4-party DeMUX augments its 2-party counterpart with a probabilistic *sanity check* consisting of two main components that respectively check that:

1. $\text{flags}(\overline{\text{seed}}_0) \oplus \text{flags}(\overline{\text{seed}}_1) = \vec{e}_j$ for some $j \in [1..r]$, and
2. the leaf-layer CW shares $(\delta, cw_h - \delta)$ are correct.

Both components borrow well-known ideas from batch testing [6], although the way they implement these ideas as 4-party interactive protocols appears to be new. The components of the sanity check run in tandem; however, for clarity and ease of exposition, we treat each step in isolation.

6.3.2.1 Testing the flag bits

The first component of the sanity check tests if $\text{flags}(\overline{\text{seed}}_0) \oplus \text{flags}(\overline{\text{seed}}_1) = \vec{e}_j$ for some $j \in [1..r]$; in other words, it checks if $\overline{\text{seed}}_0$ and $\overline{\text{seed}}_1$ together define a valid 2-party MUX (see §6.2.1). Intuitively, the idea is to have P_3 select a vector $\vec{R} \in_{\mathrm{R}} [0..2^{\mu} - 1]^r$, which it sends to P_0 and P_1; then, treating $\vec{R}$ a "random database", P_0 and P_1 respectively use $\text{flags}(\overline{\text{seed}}_0)$ and $\text{flags}(\overline{\text{seed}}_1)$ as queries in a 2-server instance of the celebrated Chor–Gilboa–Kushilevitz–Sudan (CGKS) PIR protocol [15], forwarding the results for P_3 to reconstruct. If the CGKS responses provided to P_3 reconstruct to a component of $\vec{R}$, then the parties accept $\overline{\text{seed}}_0$ and $\overline{\text{seed}}_1$ as valid.

The problem with this simplistic protocol is, of course, that it leaks to P_3 the index j of the standard basis vector $\vec{e}_j$. To prevent this, P_0 and P_1 enlist the help of P_2, who selects a blinding factor γ and a random cyclic permutation $\pi\colon [1..r] \to [1..r]$, and then sends both of these values to P_0 and P_1—but not to P_3. P_0 and P_1 use π to *cyclically shift* the components of $\text{flags}(\text{seed}_0)$ and $\text{flags}(\text{seed}_1)$ before using them as CGKS queries, and then they blind the CGKS responses with γ before sending them to P_3. Intuitively, the permutation π severs the link between the index j and the matching component $\vec{R}[\pi(j)]$, while γ prevents P_3 from inferring π using $\overline{\text{seed}}_1$—which it also knows—in conjunction with $\vec{R}$ and the response it receives from P_1. Figure 3 illustrates this component of the sanity check. Note that the same batch-testing vector $\vec{R}$ can be used to check arbitrarily many DPFs, provided (i) $\vec{R}$ is sampled after the DPFs are generated and (ii) each run of the test uses fresh choices for π, γ.

A simple derivation confirms that P_3 accepts whenever $\overline{\text{seed}}_0$ and $\overline{\text{seed}}_1$ are indeed well-formed (leafless) DPF seeds, as output by $\overline{\text{Gen}}$. The next observation follows from a standard counting argument [34, Appendix A].

Observation 2. *Let* $\overline{\text{seed}}_0$ *and* $\overline{\text{seed}}_1$ *be fixed, and let* $\vec{R} \in_{\mathrm{R}} [0..2^{\mu} - 1]^r$. *If* $\text{flags}(\overline{\text{seed}}_0) \oplus \text{flags}(\overline{\text{seed}}_1) \neq \vec{e}_j$, *then* $\Pr\big[\langle \text{flags}(\overline{\text{seed}}_0), \vec{R} \rangle \oplus \langle \text{flags}(\overline{\text{seed}}_1), \vec{R} \rangle = \vec{R}[j]\big] = 2^{-\mu}$.

Corollary 1. *If* $\text{flags}(\overline{\text{seed}}_0) \oplus \text{flags}(\overline{\text{seed}}_1)$ *is not a standard basis vector, then* P_3 *rejects with probability at least* $1 - r/2^{\mu}$.

We remark Boyle et al. propose a 2-party DPF sanity check [10, §4] based on linear sketching and 2PC, while Corrigan-Gibbs, Boneh, and Mazières propose a 3-party DPF sanity check based on hashing [16, §5.1]. The availability of a fourth party enables us achieve comparable soundness error using a much simpler—and faster—protocol.

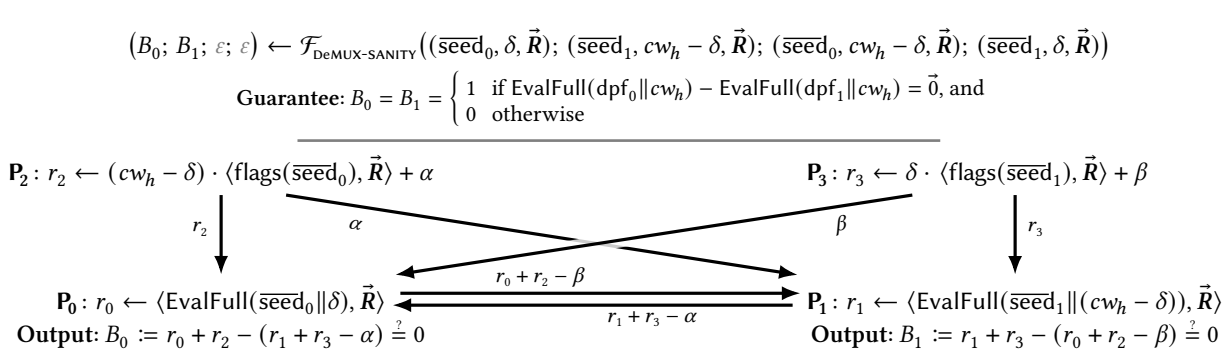


Fig. 4. Second component of 4-party DPF sanity check, as described in Section 6.3.2.2.

6.3.2.2 Testing the leaf-layer CW shares

The second component of the sanity check employs the same "mix-and-match" pattern as the 4-party MUX (§6.2.2) to test if the leaf-layer CW shares are well formed. Suppose P_0 and P_1 respectively hold $(\overline{\text{seed}}_0, \delta)$ and $(\overline{\text{seed}}_1, cw_h - \delta)$, while P_2 and P_3 respectively hold $(\overline{\text{seed}}_0, cw_h - \delta)$ and $(\overline{\text{seed}}_1, \delta)$; the goal is to verify that indeed

$$\vec{0} \stackrel{?}{=} \text{EvalFull}(\overline{\text{seed}}_0 \| cw_h) - \text{EvalFull}(\overline{\text{seed}}_1 \| cw_h) . \tag{12}$$

By construction [10, Fig. 1], we have that

$$\begin{aligned} \text{EvalFull}(\overline{\text{seed}}_0 \| cw_h) &= \text{EvalFull}(\overline{\text{seed}}_0 \| \vec{0}) + cw_h \, \text{flags}(\overline{\text{seed}}_0) \\ &= \text{EvalFull}(\overline{\text{seed}}_0 \| \delta) \\ &\quad + (cw_h - \delta) \, \text{flags}(\overline{\text{seed}}_0) , \end{aligned}$$

and, symmetrically, that

$$\begin{aligned} \text{EvalFull}(\overline{\text{seed}}_1 \| cw_h) &= \text{EvalFull}(\overline{\text{seed}}_1 \| (cw_h - \delta)) \\ &\quad + \delta \, \text{flags}(\overline{\text{seed}}_1) ; \end{aligned}$$

hence, Equation (12) holds if and only if

$$\begin{aligned} \vec{0} \stackrel{?}{=} \text{EvalFull}(\overline{\text{seed}}_0 \| \delta) \; &+ \; (cw_h - \delta) \, \text{flags}(\text{seed}_0) \\ - \text{EvalFull}(\overline{\text{seed}}_1 \| (cw_h - \delta)) \; &- \; \delta \, \text{flags}(\text{seed}_1) . \end{aligned} \tag{13}$$

Disclosing the four vectors comprising Equation (13) to a single party would leak both cw_h and the value on the selection wire—to say nothing of the prohibitively high communication overhead. Instead, each of the four parties computes the inner product of their respective vectors with a random vector $\vec{R}$ and then they use a 2-move protocol to check that the resulting inner products sum to 0. Note that the same random vector $\vec{R}$ can be (i) shared among the two components of the test and (ii) reused to test an arbitrary number of DPFs. Figure 4 illustrates this component of the sanity check.

6.4 4PC fixed-point vector normalization

Recall that user profiles $\vec{u}_i$ and item profiles $\vec{v}_j$ are length-d unit vectors of fixed-point numbers with fractional precision p; thus, the adaptation values $\Delta_{u_i} = \vec{v}_j(1 - \langle \vec{u}_i, \vec{v}_j \rangle)$ and $\Delta_{v_j} = \vec{u}_j(1 - \langle \vec{u}_i, \vec{v}_j \rangle)$ are themselves at-most-unit magnitude vectors[5] having fractional precision $3p$. Profile updation consists of summing one or more such adaptation values into a given profile. As detailed in Section 5.2, we choose p to maximize fixed-point approximation accuracy subject to an "overflow quota" allowing the accumulation of some tuneable number of adaptation values without risk of overflowing into the most-significant bits of profile components.

For user-profile updation, the servers know precisely how many adaptation values are accumulated; for item-profile updation, the DeMUX hides this number and we must count every adaptation value against the overflow quota of *all* item profiles. In both cases, upon hitting the overflow quota, the servers must perform a precision reduction, as described in Section 6.4.1, to allocate additional bits for the integer parts of the profile components.

6.4.1 3-party precision reduction

Suppose P_0 holds x_0 and P_1 holds x_1 such that $x = x_0 + x_1$ is a fixed-point number with precision P. We now describe a protocol using which P_0 and P_1 (with some help from P_2) can produce shares of the largest fixed-point number less than or equal to x having precision $P - s$, where $s \in [1..P]$ is a parameter. If x were given in cleartext, such a precision reduction would be trivially realized by an arithmetic shift to the right by s bits; in the $(2, 2)$-additive sharing setting, by contrast, we must account for any implicit reductions that arise when evaluating $x = x_0 + x_1$ as 64-bit unsigned integers.

Our approach assumes—nay, insists—that the most-significant bit of x is always 0, and then it examines the most-significant bits of x_0 and x_1 to identify three cases:

1. $\text{msb}(x_0) = \text{msb}(x_1) = 0$ so that $x = x_0 + x_1$ over $\mathbb{Z}$, in which case no implicit reduction occurs and no corrective action is needed;
2. $\text{msb}(x_0) \neq \text{msb}(x_1)$ so that $x = x_0 + x_1 + 2^{64}$ over $\mathbb{Z}$, in which case the arithmetic shifts of x_0 and x_1 preserve a carry chain that results in the desired implicit reduction and, again, no corrective action is needed; and
3. $\text{msb}(x_0) = \text{msb}(x_1) = 1$ so that $x = x_0 + x_1 - 2^{64}$ over $\mathbb{Z}$, in which case the arithmetic shifts produce shares of a secret having its s most-significant bits (incorrectly) set.

5 That Δ_{u_i} and Δ_{v_j} have magnitude at most 1 is an immediate consequence of the Cauchy–Schwarz inequality.

$$\big((\bar{U}_0, sign_1);\ (\bar{U}_1, sign_0);\ \varepsilon;\ \varepsilon\big) \leftarrow \mathcal{F}_{\mathsf{ISQRT}}\big((U_0, \mathsf{coeffs});\ (U_1, \mathsf{coeffs});\ (U_0, \mathsf{coeffs});\ (U_1, \mathsf{coeffs})\big)$$

Guarantee: $U_0 + U_1 < 2^{15} \implies sign_0\bar{U}_0 + sign_1\bar{U}_1 \approx (U_0 + U_1)^{-1/2}$

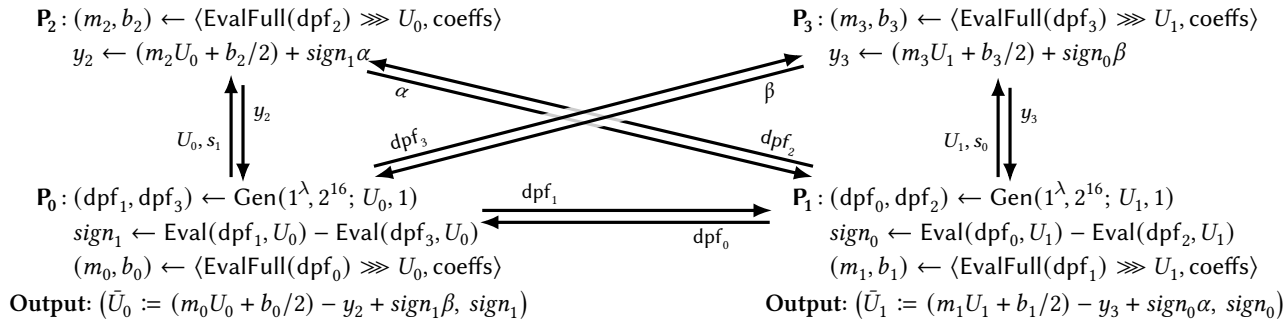


Fig. 5. 4-party inverse square-root protocol, as described in Section 6.4.2. In the diagram, '$\ggg$' is the cyclic right shift operator.

Given these three cases, it suffices to add $(2, 2)$-additive shares of 2^{64-s} to the arithmetic shifts of x_0 and x_1 if and only if $\mathrm{msb}(x_0) = \mathrm{msb}(x_1) = 1$, a procedure we can implement obliviously by computing the product of $\mathrm{msb}(x_0)$ and $\mathrm{msb}(x_1)$ using a 1-bit Du–Atallah multiplication and the help of P_2, and then scaling the resulting shares by 2^{64-s}.

6.4.2 Normalizing the precision-reduced vectors

Suppose P_0 and P_1 respectively hold $(2, 2)$-additive shares u_0' and u_1' of an updated, but not-yet-normalized profile $\vec{\boldsymbol{u}}_i{}'$ with fractional precision p. Their goal is to arrive at shares u_0 and u_1 of $\vec{\boldsymbol{u}}_i := c \cdot \vec{\boldsymbol{u}}_i{}'$, where $c \approx 1/\|\vec{\boldsymbol{u}}_i{}'\|_2$. To do this, they enlist the help of P_2 and P_3 in a 4-party protocol. All four parties hold in common an associative array coeffs mapping ranges of possible values of $\|\vec{\boldsymbol{u}}_i{}'\|_2^2$ to coefficients for a good linear approximation to *inverse square roots* within that range. (Our implementation employs a simple greedy algorithm to find ranges and coefficients exhibiting low approximation errors.)

At the outset, P_0 and P_1 compute shares U_0 and U_1 of $\langle \vec{\boldsymbol{u}}_i{}', \vec{\boldsymbol{u}}_i{}'\rangle$ using generalized Du–Atallah (and help from P_2). From here, the four parties employ a modified PIR protocol to fetch suitable coefficients from coeffs, as illustrated in Figure 5: P_0 sends U_0 to P_2 and also "reshares" it with P_1 and P_3 as the target of a $(2, 2)$-DPF with point $(U_0, 1) \in \mathbb{Z}_{|R|} \times \mathbf{GF}(2)$. P_1 likewise sends U_1 to P_3 and "reshares" it with P_0 and P_2 as the target of a $(2, 2)$-DPF with point $(U_1, 1) \in \mathbb{Z}_{|R|} \times \mathbf{GF}(2)$. P_0 and P_2 now hold $U_0 \in \mathbb{Z}_{|R|}$ and, by way of their DPF seeds, $(2, 2)$-additive shares of $\vec{\boldsymbol{e}}_{U_1} \in \mathbb{Z}^{|R|}$; they each invoke EvalFull on their respective DPF seeds, and then *cyclically shift* the output by U_0. This effectively moves the target point of the DPF to $(U_0 + U_1, 1) \in \mathbb{Z}_{|R|} \times \mathbf{GF}(2)$. P_1 and P_3 likewise compute shares of $\vec{\boldsymbol{e}}_{U_1+U_0}$ using U_1 and their DPF seeds from P_0.

Finally, both pairs evaluate their shares of $\vec{\boldsymbol{e}}_{U_1+U_0}$ as CGKS queries against coeffs, yielding shares of $\pm(m, b)$ such that $m(U_0+U_1)+b$ is a good linear approximation for $(U_0+U_1)^{-1/2} = 1/\|\vec{\boldsymbol{u}}_i{}'\|_2$. Taking advantage of linearity, P_0 and P_2 compute $y_0 := m \cdot U_0 + b/2$ while P_1 and P_3 compute $y_1 := m \cdot U_1 + b/2$. A final Du–Atallah multiplication scales $\vec{\boldsymbol{u}}_i$ by $y_0 + y_1$.

To improve efficiency in the DPF steps, our implementation truncates U_0 and U_1 to 6 bits fractional precision (and we support at most 9 bits for the integer part), yielding shares in $R = \mathbb{Z}_{2^{16}}$; we still compute y_0 and y_1 using all 64 bits of U_0 and U_1. Even with this optimization, we can always approximate inverse square roots with relative error less than 10^{-5}.

7 Providing recommendations

We now describe our 3PC for providing recommendations based on user profile $\vec{\boldsymbol{u}}_i$ and item-profile matrix $\boldsymbol{V} = \big[\vec{\boldsymbol{v}}_1; \cdots; \vec{\boldsymbol{v}}_r\big]$. The strategy, at a high level, is to sample recommendations via the discrete probability distribution whose *probability mass function* is given by (normalizing) the vector $pmf_i := \langle p_{i,1}, \ldots, p_{i,r}\rangle$ in which each $p_{i,j} \in [0, 1]$ is the inner product between $\vec{\boldsymbol{u}}_i$ and $\vec{\boldsymbol{v}}_j$.

To this end, P_0 and P_1 enlist the help of P_2 to compute shares of pmf_i using r parallel generalized Du-Atallah invocations. Next, they non-interactively transform their pmf_i shares into shares of a (scaled) *cumulative distribution function* by taking partial sums; that is, they compute shares of

$$cdf_i := \left\langle p_{i,1}, p_{i,1} + p_{i,2}, \ldots, \textstyle\sum_{j=1}^{r} p_{i,j}\right\rangle.$$

To sample from the distribution, it now suffices for P_0 and P_1 to choose a uniform $x \in_{\mathrm{R}} [0, \sum_{j=1}^{r} p_{i,j}]$ and then return the index j of the smallest number in cdf_i that is greater than or equal to this x. For this last step, they employ a novel 3PC method to evaluate the greater-than-or-equal-to operator (over $\mathbb{Z}$) on $(2, 2)$-additive shares.

7.1 3-party integer comparison

The task of MPC-based integer comparison has received considerable attention. Most prior works (e.g., [9, 17, 26, 50]) implement comparison using (inherently costly) *bit decomposi-*

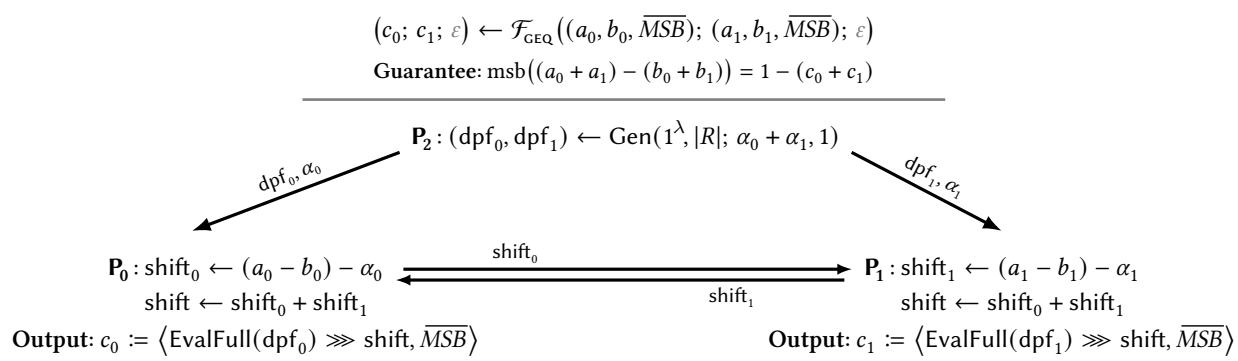


Fig. 6. 3-party comparison protocol, as described in Section 7.1. In the diagram, '$\ggg$' is the cyclic right shift operator.

tion as a subroutine. One notable exception is the constant-round construction of Nishide and Ohta [50]; however, their construction is specific to secret sharing over *odd-order* rings and is, consequently, not applicable to our setting.

Our new 3PC comparison protocol is reminiscent of our 4-party fixed-point vector normalization protocol (§6.4.2). Specifically, our method checks whether $a \geq b$ over $\mathbb{Z}$ by using a modified PIR protocol to map the difference $a - b$ to its most-significant bit, which will equal 1 if and only if $a - b$ overflows (which happens if and only if $a < b$).

Suppose P_0 and P_1 respectively hold (a_0, b_0) and (a_1, b_1) such that $a_0 + a_1 = a$ and $b_0 + b_1 = b$; the goal is for P_0 and P_1 to output shares of 1 if $a \geq b$ and of 0 otherwise. To this end, consider the "database" $\overline{MSB} = 1^{|R|/2} \| 0^{|R|/2} \in \mathbf{GF}(2)^{|R|}$ mapping each possible value of $a - b \in R$ to $1 - \mathrm{msb}(a, b)$. From here, the three parties employ a modified PIR protocol to fetch the answer from $\overline{MSB}$, as illustrated in Figure 6: P_2 samples $(\mathsf{dpf}_0, \mathsf{dpf}_1) \leftarrow \mathsf{Gen}(1^\lambda, |R|;\, \alpha_0 + \alpha_1, 1)$ for uniform $\alpha_0, \alpha_1 \in_{\mathrm{R}} R$, and then it sends $(\mathsf{dpf}_0, \alpha_0)$ to P_0 and $(\mathsf{dpf}_1, \alpha_1)$ to P_1. Next, P_0 sends $\mathsf{shift}_0 := (a_0 - b_0) - \alpha_0 \in R$ to P_1 and P_1 sends $\mathsf{shift}_1 := (a_1 - b_1) - \alpha_1 \in R$ to P_0, and then both parties compute

$$\begin{aligned}\mathsf{shift} &= \mathsf{shift}_0 + \mathsf{shift}_1 \\ &= (a_0 - b_0 - \alpha_0) + (a_1 - b_1 - \alpha_1) \\ &= (a_0 + a_1) - (b_0 - b_1) - (\alpha_0 + \alpha_1) \\ &= (a - b) - (\alpha_0 + \alpha_1)\,.\end{aligned}$$

Finally, P_0 and P_1 respectively compute the full-domain evaluations of dpf_0 and dpf_1, cyclically shifting by shift, and then they output its inner product with $\overline{MSB}$. Similar to the normalization protocol, the cyclic shift effectively moves the DPF target from $\alpha_0 + \alpha_1$ to $a - b$, while the inner product maps the result to 1 if the most-significant bit of this new target is 0, and to 0 otherwise.

7.2 Optimization

In the case of PIRSONA, we know upper bounds on each of the integers to be compared. Specifically, must compare $a \in_{\mathrm{R}} [0, N]$, where $N = \sum_{j=1}^{r} p_{i,j} \leq r$, with each of $p_{1,i} \leq 1$, $p_{1,i} + p_{2,i} \leq 2$, and so on, up to $\sum_{j=1}^{r} p_{i,j} \leq r$.

Here we briefly sketch an optimization that performs this sequence of comparisons efficiently by taking advantage of the known upper bounds; full details of the optimization are given in Appendix A. Suppose we know *a priori* that $a \in [0, 2^m]$ and $b \in [0, 2^n]$ for some $m > n$. The idea is to rewrite a as $a_{\mathsf{hi}} \| a_{\mathsf{lo}}$, where a_{hi} is the high-order $m - n$ bits and a_{lo} the low-order n bits of a. There are now two cases to consider:

1. $a_{\mathsf{hi}} > 0$, in which case $a \geq b$ irrespective of a_{lo}; and
2. $a_{\mathsf{hi}} = 0$, in which case $a \geq b$ if and only if $a_{\mathsf{lo}} \geq b$.

The idea is to "compress" a_{hi} into a single bit x (where $x = 0$ if $a_{\mathsf{hi}} = 0$ and $x = 1$ otherwise) using an efficient 3PC protocol, and then to apply the DPF-based comparison protocol to $x \| a_{\mathsf{lo}} \in [0, 2^{n+1}]$ and $b \in [0, 2^n]$.

8 Implementation & evaluation

In order to validate the practicality of PIRSONA, we wrote a proof-of-concept implementation in C++ using the open-source library dpf++ [35] for $(2, 2)$-DPFs and Asio for asynchronous socket communication [40].[6]

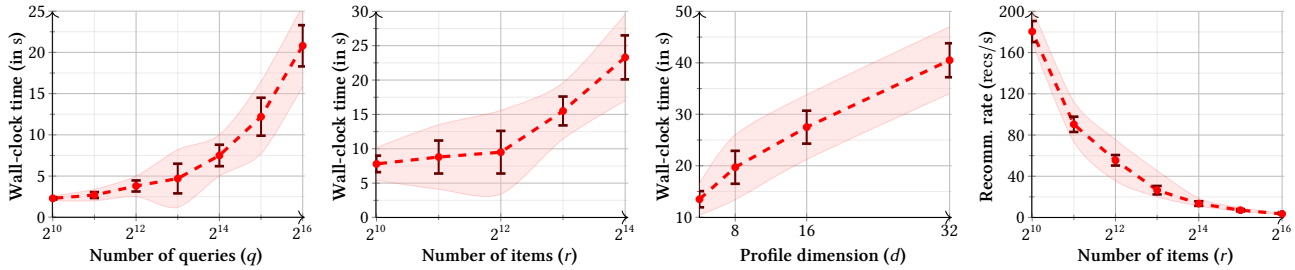
Experimental setup

We ran PIRSONA on Amazon EC2 servers running in four geographically distant regions; namely, we ran P_0 in the us-west-2 region (Oregon), P_1 in the us-east-1 region (North Virgina), P_2 in the eu-west-1 region (Ireland), and P_3 in the us-east-2 region (Ohio). Each party ran on its own m5.2xlarge instance (8 vCPUs and 32GiB of RAM) running the standard Ubuntu 18.04 LTS AMI.

Most experiments use *MovieLens* [33], a standard dataset collection used as a benchmark by the collaborative filtering research community. Specifically, we used two datasets from *MovieLens*, each consisting of $q = 100\,000$ queries:

1. **ML-100K** with $m = 943$ users and $r = 1\,700$ items; and
2. **ML-Latest** with $m = 610$ users and $r = 9\,000$ items.

6 Our source code is available for anonymous review via https://github.com/pirsona/PIRSONA; it will be made publicly available under the GNU General Public License version 3 upon acceptance of this paper.



(a) Varying the number of queries from $m = 943$ users over $r = 2^{12}$ items using profiles of dimension $d = 8$.

(b) Varying the number of items with $q = 2^{14}$ queries from $m = 943$ users using profiles of dimension $d = 8$.

(c) Varying the profile dimension with $q = 2^{16}$ queries from $m = 943$ users over $r = 2^{10}$ items.

(d) Recommendation rate (per s) as the number of items varies using profiles of dimension $d = 8$.

Fig. 7. Wall-clock training time and recommendation rate for scalability microbenchmarks. Error bars show ± one standard deviation; the shaded bands are 95% confidence intervals ($z = 1.96$). Notice that the x-axes of Figures 7a, 7b, and 7d use a log scale with base 2.

The first dataset is the same one used by Nikolaenko, Ioannidis, Weinsberg, Joye, Taft, and Boneh [48] in the context of garbled circuit–based collaborative filtering, which is perhaps the most closely related work in the literature. The second dataset is similar, but it includes over a factor 5× more items, which greatly affects the performance of our DPF-based MUX and DeMUX constructions. We remark that data in the *MovieLens* datasets contain explicit *numerical ratings*; however, as we deal exclusively with collaborative filtering based on Boolean matrix factorization, we replace all numeric ratings with the value 1. To measure how PIRSONA scales with a varying number of queries, profile dimensions, or items in the database, we also ran a handful of experiments over synthetic (random) data of varying sizes.

Each of our experiments measures the (wall-clock) training times for a given dataset, which includes both processing time and network latency. We repeated all experiments for 100 trials and our plots and tables report the sample mean running timings across those 100 trials, with error bars indicating one standard deviation from the sample mean. In tables and when discussing selected statistics in the text, we report all figures to one digit of precision in the sample standard deviation. Except where otherwise specified, all experiments were run with $p = 18$ bits of precision for the fractional part of user- and item-profile components, leaving $64 - 3 \cdot 18 - 1 = 9$ bits for integer parts when accumulating adaptation values.

8.1 Bandwidth consumption

Before presenting the results of our running-time experiments, we briefly analyze the bandwidth consumption of PIRSONA. Recall that P_2 sends mostly random blinding factors and cancellation terms to P_0 and P_1; as the DPFs already assume the existence of PRGs, PIRSONA reduces the communication cost by having P_2 send λ-bit PRG seeds to P_0 and P_1 so that they can generate blinding factors locally. The total server-to-server bandwidth consumption, accounting for this optimization, is summarized in Table 1.

Table 1. Communication cost (in bytes) for q queries and x normalizations when profiles have dimension d.

Sender \ Receiver	P_0	P_1	P_2	P_3
P_0	—	$q(24d+24)+2^9\lambda x$	$2x$	$8q+2^9\lambda x$
P_1	$q(24d+24)+2^9\lambda x$	—	$2^9\lambda x$	$8q+2x$
P_2	$2\lambda+q(32d+24)+16x$	$4\lambda+q(16d+24)+16x$	—	—
P_3	$4\lambda+16x$	$\lambda+16qd+16x$	—	—

8.2 Scalability microbenchmarks

Our first running-time experiments measure the training time of PIRSONA over synthetic (random) data, with the goal of isolating the effect of varying a single parameter when all others are fixed. To this end, we ran three sets of experiments respectively measuring the impact on training time of varying (i) the number of queries being trained on, (ii) the number of items r in the database, and (iii) the dimension d of the profiles. We also measured the rate at which PIRSONA can render recommendations while varying the number of items r.

Varying the number of queries

Figure 7a plots the (wall-clock) training time for $m = 943$ users, $r = 2^{12}$ items, and profiles of dimension $d = 8$ as the number of queries ranges from $q = 2^{10}$ up to 2^{16}. As expected, the measured training times scale linearly (linear regression gives $R^2 = 0.9972$) from about 2.3 ± 0.1s for $q = 2^{10}$ queries up to 21 ± 3s for $q = 2^{16}$ queries (an effective rate of about 3 500 queries/s).

Varying the number of items

Figure 7b plots the (wall-clock) training time for $q = 2^{14}$ queries, $m = 943$ users, and profiles of dimension $d = 8$ as the number of items ranges from $r = 2^8$ up to 2^{14}. Again we observe the expected linear scaling (linear regression gives $R^2 = 0.9889$) from about 8 ± 1s for $r = 2^8$ items through 28 ± 3s for $r = 2^{14}$ items (≈ 1s slowdown per 1000 items).

Varying the profile dimensions

Figure 7c plots the (wall-clock) training time for $q = 2^{16}$ queries, $m = 943$ users, and $r = 2^{10}$ items as the dimension of user and item profiles ranges from $d = 4$ up to 32. Again, the measured training times (after $d = 8$) scale linearly (linear regression gives $R^2 = 0.9978$) from about 20 ± 3s for $d = 8$ up to 41 ± 3s for $d = 32$ (≈ 0.9s slowdown per profile component). The initially steeper slope between $d = 4$ and 8 results from the former profiles occupying a single 256-bit AVX2 register and the latter spanning multiple registers.

Recommendation rate

Figure 7d plots the number of recommendations rendered per second for profiles of dimension $d = 8$ as the number of items ranges from $r = 2^{10}$ up to 2^{16}. We observe slightly superlinear scaling as the number of items grows (owing to the optimization in Section 7.2) from about 180 ± 10 recommendations/s for $r = 2^{10}$ items down to 3.5 ± 0.2 recommendations/s for $r = 2^{16}$ items.

8.3 *MovieLens* experiments

Table 2 displays the training and recommendation times for PIRSONA on the two *MovieLens* datasets. Recall that both datasets include 100000 queries; thus, even the slowest experiment (**ML-Latest** with profiles of dimension $d = 8$) had mean training throughput exceeding 1400 queries/s. We also compared the training time of PIRSONA against that of our own (lightly optimized) non-private collaborative filtering implementation and found that PIRSONA ran around 150× slower. We forgo a detailed comparison because significant speedups to the non-private code are surely possible.

The rightmost column in Table 2 displays the mean time for providing one recommendation to *each* user, of which there are $m = 943$ in **ML-100K** and $m = 610$ in **ML-Latest**. For the **ML-100K** dataset, this works out to 38 ± 4 recommendations/s when profiles have dimension $d = 4$ and 45 ± 5 recommendations/s when they have dimension $d = 8$; for the **ML-Latest** dataset, these rates decrease to 24 ± 4 recommendations/s when profiles have dimension $d = 4$ and 22 ± 5 recommendations/s when they have dimension $d = 8$, due to the comparatively large number of items ($r = 9000$ versus $r = 1700$).

Table 2. Wall-clock running times for *MovieLens*. The rightmost column is time to make m recommendations (one per user).

Data Set	Profile dims.	Grad. descent	User prof. normalize	Item prof. normalize	Recomm. m items
ML-100K	4	22 ± 4s	0.7 ± 0.4 s	1.8 ± 0.1s	21 ± 4s
	8	32 ± 6s	0.95 ± 0.09s	2.0 ± 0.2s	24 ± 3s
ML-Latest	4	49 ± 4s	0.53 ± 0.08s	3.5 ± 0.4s	26 ± 3s
	8	65 ± 6s	0.7 ± 0.1 s	4.2 ± 0.7s	28 ± 4s

Mean squared error

PIRSONA's use of fixed-point arithmetic introduces approximation errors relative to a non-private collaborative filtering implementation using floating-point arithmetic. To quantify the effects of fixed-point approximations, we compared the *mean squared error* between the user and item profiles produced by PIRSONA on the **ML-100K** dataset relative to those produced by our non-private collaborative filtering implementation, which uses IEEE double-precision floating-point arithmetic. A similar metric was used to measure fixed-point approximation errors in the related literature [48, 49]. We also measured what effect varying the fractional precision had on training times, since higher fractional precision necessitates more frequent precision reductions; fortunately, this difference was statistically insignificant with only $p = 18$ exhibiting a slightly higher sample mean. The mean squared error was minuscule with $p = 14$ or more bits (whereas below $p = 14$ bits, it rapidly approached 1).

Table 3. Mean squared error and training time for select fractional precisions on the **ML-100K** dataset with profile dimension $d = 8$.

Fractional precision	Running time	User prof. MSE	Item prof. MSE
18	35 ± 8s	4×10^{-5}	4.1×10^{-5}
16	32 ± 6s	2×10^{-5}	1.8×10^{-5}
14	32 ± 6s	1.5×10^{-4}	1.7×10^{-4}
12	32 ± 6s	1.4×10^{-2}	1.5×10^{-2}

8.4 Comparison with garbled circuits

The most closely related prior work is detailed in a manuscript by Nikolaenko, Ioannidis, Weinsberg, Joye, Taft, and Boneh [48]. They use *garbled circuits* to train similar collaborative filtering models as PIRSONA. Fundamental model differences render an apples-to-apples comparison impossible, yet it is instructive to note that Nikolaenko et al. report 2.9 hours to train on 14,683 queries from the **ML-100K** dataset—a full *five orders of magnitude* slower than PIRSONA.

9 Related literature

There does not appear to be any prior work specifically addressing the problem recommendations in the context of PIR-based systems; however, past efforts have addressed other related problems. We broadly divide these into three categories: private recommender systems, privacy-preserving e-commerce, and privacy-preserving targeted advertising.

9.1 Private recommender systems

A number of prior works have studied private recommender systems in a wide variety of cryptographic contexts.

Garbled circuits–based systems
Yao's *garbled circuits* [58] is a cryptographic protocol to do two-party secure computation. Nikolaenko, Ioannidis, Weinsberg, Joye, Taft, and Boneh [48] constructed a private recommender system by using garbled circuits to do gradient descent. Garbled circuits have also been used to do privacy-preserving ridge regression by Nikolaenko, Weinsberg, Ioannidis, Joye, Boneh, and Taft [49], which can also be used to build recommender engines.

Trusted hardware–based systems
Trusted hardware like Intel's *Software Guard eXtensions* (SGX) have been used build recommender systems; for example, in a series of papers by Aïmeur, Brassard, Fernandez [2, 3]. Ohrimenko, Schuster, Fournet, Mehta, Nowozin, Vaswani, and Costa [51] perform various machine learning algorithms like support vector machines, neural networks, k-means clustering using trusted SGX-processors.

Differentially private recommendations
McSherry and Mironov [45] develop differentially private recommender systems. Liu, Wang, and Smola [44] performed collaborative filtering using differentially private matrix factorization.

Fully homomorphic encryption
Kim, Kim, Koo, Kim, Yoon, and Shin [39] use fully homomorphic encryption to perform privacy-preserving matrix factorization. Canny [12] also uses homomorphic encryption for collaborative filtering. Polat and Du [53] use homomorphic encryption to do SVD-based collaborative filtering. Erkin, Veugen, Toft, and Lagendijk [22] also provide private recommendations using homomorphic encryption.

Other Contexts
Lam and Riedl [43] and Mobasher, Burke, Bhaumik, and Williams [47] have studied recommender engines in the context of "shilling attacks". Polat and Du [54] consider SVD-based collaborative filtering that achieves privacy via random perturbations. Canny [13] provides a method of doing collaborative filtering using factor analysis.

9.2 Media consumption and e-commerce

The surge in online media consumption has meant an increased risk of losing privacy for its consumers, which has recently spurred the development of several privacy-preserving media consumption platforms. PriView [7] is an example of an online media consumption system that provides recommendations by distorting user ratings. Gupta, Crooks, Mulhern, Setty, Alvisi, and Walfish [29] describe a PIR-based media delivery system called Popcorn.

Much like online-media consumption, e-commerce also poses a threat to the users' privacy. Egger [21] studies e-commerce during its inception days from a Human-Computer Interaction point of view. Ackerman, Cranor, and Reagle [1] study privacy in e-commerce. Henry, Olumofin, and Goldberg [34] use information-theoretic PIR to enable privacy-preserving e-commerce.

9.3 Privacy-preserving ad targetting

An increase in online media consumption and e-commerce has provided service providers an incentive to do targeted advertising. Targeting advertising is a form of online advertising where specific advertisements are directed towards specific users who are likely to have an interest in that product. Mhaidli, Zou, and Schaub [46] conducted a series of interviews with app developers, which reveal that they see targeted advertising as the only viable way to monetize their app. This, of course, comes at the cost of users' privacy, thus motivating research on privacy-preserving targeting advertising. Juels [37] provides privacy on targeted advertising using PIR. AdROSA [38], Privad [28], Farahat [23] are other solutions to do targeted advertising with privacy. TrackMeNot (TMN) [57] is another system to do secure targeted advertising. TMN obfuscates the users' queries by programmatically generating several fake queries which mimic the user behavior. Hardt and Nath [32] provide a solution based on differential privacy. Toubiana, Narayanan, Boneh, Nissenbaum, and Baroca [56] via their system Adnostic show that it is possible to support targeted advertising without compromising user privacy. Their solution relies on the browser to locally process the history database to determine user interests. Green, Ladd, and Miers [27] build on Adnostic. Their system scales to billions of ad impressions. Adsplit [55] manages user permissions to provide security against targeted advertising.

10 Concluding remarks

We presented PIRSONA, a digital content delivery system that realizes collaborative-filtering recommendations atop private information retrieval (PIR). This combination of seemingly antithetical primitives makes possible–for the first time–the construction of e-commerce and digital media delivery systems that can provide personalized content recommendations based on their users' historical consumption patterns while simultaneously keeping said consumption patterns private. A series of experiments in our proof-of-concept implementation suggest that PIRSONA can easily scale to systems serving thousands of users per second.

Toward building PIRSONA, we proposed several 4PC primitives of independent interest, including (i) efficient 2- and 4PC fixed–selection wire multiplexers (MUXs) and demultiplexers (DeMUXs) from $(2, 2)$-distributed point functions, (ii) fast 4PC normalization for secret-shared fixed-point vectors, (iii) 4PC well-formedness tests for $(2, 2)$-distributed point functions, and (iv) a novel, constant-round 3PC integer ("less than") comparison.

References

[1] Mark S. Ackerman, Lorrie Faith Cranor, and Joseph Reagle. Privacy in e-commerce: Examining user scenarios and privacy preferences. In *Proceedings of EC 1999*, pages 1–8, Denver, CO, USA (November 1999).

[2] Esma Aïmeur, Gilles Brassard, José Manuel Fernandez, and Flavien Serge Mani Onana. Privacy-preserving demographic filtering. In *Proceedings of ACM SAC 2006*, pages 872–878, Dijon, France (April 2006).

[3] Esma Aïmeur, Gilles Brassard, José Manuel Fernandez, and Flavien Serge Mani Onana. ALAMBIC: A privacy-preserving recommender system for electronic commerce. *International Journal of Information Security (IJIS)*, 7(5):307–334, October 2008.

[4] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology: Proceedings of CRYPTO 1991*, volume 576 of *LNCS*, pages 420–432, Santa Barbara, CA, USA (August 1991).

[5] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers' computation in private information retrieval: PIR with preprocessing. *Journal of Cryptology*, 17(2):125–151, March 2004.

[6] Mihir Bellare, Juan A. Garay, and Tal Rabin. Fast batch verification for modular exponentiation and digital signatures. In *Advances in Cryptology: Proceedings of EUROCRYPT 1998*, volume 1403 of *LNCS*, pages 236–250, Espoo, Finland (June 1998).

[7] Sandilya Bhamidipati, Nadia Fawaz, Branislav Kveton, and Amy Zhang. PriView: Personalized media consumption meets privacy against inference attacks. *IEEE Software*, 32(4):53–59, July–August 2015.

[8] Simon R. Blackburn, Tuvi Etzion, and Maura B. Paterson. PIR schemes with small download complexity and low storage requirements. In *Proceedings of ISIT 2017*, pages 146–150, Aachen, Germany (June 2017).

[9] Dan Bogdanov, Sven Laur, and Jan Willemson. Sharemind: A framework for fast privacy-preserving computations. In *Proceedings of ESORICS 2008*, volume 5283 of *LNCS*, pages 192–206, Málaga, Spain (October 2008).

[10] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of CCS 2016*, pages 1292–1303, Vienna, Austria (October 2016).

[11] Joseph A. Calandrino, Ann Kilzer, Arvind Narayanan, Edward W. Felten, and Vitaly Shmatikov. "You might also like:" Privacy risks of collaborative filtering. In *Proceedings of IEEE S&P 2011*, pages 231–246, Berkeley, CA, USA (May 2011).

[12] J. Canny. Collaborative filtering with privacy. In *Proceedings 2002 IEEE Symposium on Security and Privacy*, pages 45–57, May 2002.

[13] John Canny. Collaborative filtering with privacy via factor analysis. In *Proceedings of SIGIR 2002*, pages 238–245, Tampere, Finland (August 2002).

[14] Benny Chor, Niv Gilboa, and Moni Naor. Private information retrieval by keywords. Technical Report CS 0917, Technion-Israel Institute of Technology, Haifa, Israel, February 1997.

[15] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *Proceedings of FOCS 1995*, pages 41–50, Milwaukee, WI, USA (October 1995).

[16] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *Proceedings of IEEE S&P 2015*, pages 321–338, San Jose, CA, USA (May 2015).

[17] Ivan Damgård, Matthias Fitzi, Eike Kiltz, Jesper Buus Nielsen, and Tomas Toft. Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation. In *Proceedings of TCC 2006*, volume 3876 of *LNCS*, pages 285–304, New York, NY, USA (March 2006).

[18] Jack Doerner and abhi shelat. Scaling ORAM for secure computation. In *Proceedings of CCS 2017*, pages 523–535, Dallas, TX, USA (October–November 2017). ACM.

[19] Rafael Dowsley, Jeroen van de Graaf, Davidson Marques, and Anderson C. A. Nascimento. A two-party protocol with trusted initializer for computing the inner product. In *Proceedings of WISA 2010*, volume 6513 of *LNCS*, pages 337–350, Jeju Island, Korea (August 2010).

[20] Wenliang Du and Mikhail J. Atallah. Protocols for secure remote database access with approximate matching. In *E-Commerce Security and Privacy (Part II)*, volume 2 of *Advances in Information Security*, pages 87–111, February 2001.

[21] Florian N. Egger. "Trust me, I'm an online vendor": Towards a model of trust for e-commerce system design. In *Proceedings of CHI EA 2000*, pages 101–102, April 2000.

[22] Zekeriya Erkin, Thijs Veugen, Tomas Toft, and Reginald L. Lagendijk. Generating private recommendations efficiently using homomorphic encryption and data packing. *IEEE Transactions on Forensics and Security (TIFS)*, 7(3):1053–1066, June 2012.

[23] Ayman Farahat. Privacy preserving frequency capping in internet banner advertising. In *Proceedings of WWW 2009*, pages 1147–1148, Madrid, Spain (April 2009).

[24] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *Advances in Cryptology: Proceedings of EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 640–658, Copenhagen, Denmark (May 2014).

[25] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM (JACM)*, 33(4):792–807, October 1986.

[26] Ken Goss and Wei Jiang. Efficient and constant-rounds secure comparison through dynamic groups and asymmetric computations. *IACR Cryptology ePrint Archive*, Report 2018/179, February 2018.

[27] Matthew Green, Watson Ladd, and Ian Miers. A protocol for privately reporting ad impressions at scale. In *Proceedings of CCS 2016*, pages 1591–1601, Vienna, Austria (October 2016).

[28] Saikat Guha, Bin Cheng, and Paul Francis. Privad: Practical privacy in online advertising. In *Proceedings of NSDI 2011*, Boston, MA, USA (March–April 2011).

[29] Trinabh Gupta, Natacha Crooks, Whitney Mulhern, Srinath T. V. Setty, Lorenzo Alvisi, and Michael Walfish. Scalable and private media consumption with Popcorn. In *Proceedings of NSDI 2016*, Santa Clara, CA, USA (March 2016).

[30] Syed Mahbub Hafiz and Ryan Henry. Querying for queries: Indexes of queries for efficient and expressive IT-PIR. In *Proceedings of CCS 2017*, pages 1361–1373, Dallas, TX, USA (October–November 2017).

[31] Syed Mahbub Hafiz and Ryan Henry. A bit more than a bit is more than a bit better: Faster (essentially) optimal-rate many-server PIR. *Proceedings on Privacy Enhancing Technologies (PoPETS)*, 2019(4):112–131, January 2019.

[32] Michaela Hardt and Suman Nath. Privacy-aware personalization for mobile advertising. In *Proceedings of CCS 2012*, pages 662–673, Raleigh, NC, USA (October 2012).

[33] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):Article No. 19, December 2015.

[34] Ryan Henry, Femi Olumofin, and Ian Goldberg. Practical PIR for electronic commerce. In *Proceedings of CCS 2011*, pages 677–690, Chicago, IL, USA (October 2011).

[35] Ryan Henry and Adithya Vadapalli. dpf++; version 0.0.1 [computer software]. Available from: https://www.github.com/rh3nry/dpfplusplus, July 2019.

[36] Russell Impagliazzo. A personal view of average-case complexity. In *Proceedings of Structures 1995*, pages 134–147, Minneapolis, MN, USA (June 1995).

[37] Ari Juels. Targeted advertising... and privacy too. In *Proceedings of CT-RSA 2001*, volume 2020 of *LNCS*, pages 408–424, San Francisco, CA, USA (April 2001).

[38] Przemysław Kazienko and Michał Adamski. AdROSA—Adaptive personalization of web advertising. *Information Sciences*, 177(11):2269–2295, June 2007.

[39] Sungwook Kim, Jinsu Kim, Dongyoung Koo, Yuna Kim, Hyunsoo Yoon, and Junbum Shin. Efficient privacy-preserving matrix factorization via fully homomorphic encryption: Extended abstract. In *Proceedings of ASIA CCS 2016*, pages 617–628, Xi'an, China (May 2016).

[40] Christopher M. Kohlhoff. Asio; version 1.14.1 [computer software]. Available from: http://think-async.com/Asio/, August 2019.

[41] Yehuda Koren, Robert M. Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *IEEE Computer*, 42(8):30–37, August 2009.

[42] Eyal Kushilevitz and Tamer Mour. Sub-logarithmic distributed oblivious RAM with small block size. In *Proceedings of PKC 2019 (Part I)*, volume 11442 of *LNCS*, pages 3–33, Beijing, China (April 2019).

[43] Shyong K. Lam and John Riedl. Shilling recommender systems for fun and profit. In *Proceedings of WWW 2004*, pages 393–402, New York, NY, USA (May 2004).

[44] Ziqi Liu, Yu-Xiang Wang, and Alexander Smola. Fast differentially private matrix factorization. In *Proceedings of RecSys 2015*, pages 171–178, Vienna, Austria (September 2015).

[45] Frank McSherry and Ilya Mironov. Differentially private recommender systems: Building privacy into the Netflix Prize contenders. In *Proceedings of KDD 2009*, pages 627–636, Paris, France (June 2009).

[46] Abraham H. Mhaidli, Yixin Zou, and Florian Schaub. "We can't live without them!" App developers' adoption of ad networks and their considerations of consumer risks. In *Proceedings of SOUPS 2019*, Santa Clara, CA, USA (August 2019).

[47] Bamshad Mobasher, Robin Burke, Runa Bhaumik, and Chad Williams. Toward trustworthy recommender systems: An analysis of attack models and algorithm robustness. *ACM Transactions on Internet Technologies (TOIT)*, 7(4):Article No. 23, October 2007.

[48] Valeria Nikolaenko, Stratis Ioannidis, Udi Weinsberg, Marc Joye, Nina Taft, and Dan Boneh. Privacy-preserving matrix factorization. In *Proceedings of CCS 2013*, pages 801–812, Berlin, Germany (November 2013).

[49] Valeria Nikolaenko, Udi Weinsberg, Stratis Ioannidis, Marc Joye, Dan Boneh, and Nina Taft. Privacy-preserving ridge regression on hundreds of millions of records. In *Proceedings of IEEE S&P 2013*, pages 334–348, Berkeley, CA, USA (May 2013).

[50] Takashi Nishide and Kazuo Ohta. Multiparty computation for interval, equality, and comparison without bit-decomposition protocol. In *Proceedings of PKC 2007*, volume 4450 of *LNCS*, pages 343–360, Beijing, China (April 2007).

[51] Olga Ohrimenko, Felix Schuster, Cédric Fournet, Aastha Mehta, Sebastian Nowozin, Kapil Vaswani, and Manuel Costa. Oblivious multi-party machine learning on trusted processors. In *Proceedings of USENIX Security 2016*, pages 619–636, Austin, TX, USA (August 2016).

[52] Rafail Ostrovsky and Victor Shoup. Private information storage (Extended abstract). In *Proceedings of STOC 1997*, pages 294–303, El Paso, TX, USA (May 1997).

[53] Huseyin Polat and Wenliang Du. Privacy-preserving collaborative filtering on vertically partitioned data. In *Proceedings of PKDD 2005*, volume 3721 of *LNCS*, pages 651–658, Porto, Portugal (October 2005).

[54] Huseyin Polat and Wenliang Du. SVD-based collaborative filtering with privacy. In *Proceedings of ACM SAC 2005*, pages 791–795, Santa Fe, NM, USA (March 2005).

[55] Shashi Shekhar, Michael Dietz, and Dan S. Wallach. AdSplit: Separating smartphone advertising from applications. In *Proceedings of USENIX Security 2012*, pages 553–567, Bellevue, WA, USA (August 2012).

[56] Vincent Toubiana, Arvind Narayanan, Dan Boneh, Helen Nissenbaum, and Solon Barocas. Adnostic: Privacy preserving targeted advertising. In *Proceedings of NDSS 2010*, San Diego, CA, USA (February 2010).

[57] Vincent Toubiana, Lakshminarayanan Subramanian, and Helen Nissenbaum. TrackMeNot: Enhancing the privacy of web search. *arXiv:CoRR*, abs/1109.4677, September 2011.

[58] Andrew Chi-Chih Yao. Protocols for secure computations (Extended abstract). In *Proceedings of FOCS 1982*, pages 160–164, Chicago, IL, USA (November 1982).

A Optimized comparison protocol

A.1 Detecting carries out from the low bits

In this protocol we assume, that i and f are used to denote the number of bits used to represent the integer and fractional

parts of the fixed point number being compared respectively. The main idea of detecting a carry-out from $a_{\mathsf{lo},0}$ and $a_{\mathsf{lo},1}$ is that we use the DPF-based technique to detect if the msb of $a_{\mathsf{lo},0}+a_{\mathsf{lo},1}$ is set. Suppose that carry denotes the carry bit coming into hi bits. At the end of the following protocol, P_0 and P_1 would end up with carry'_0 and carry'_1 respectively, with the property that $\mathsf{carry}'_0+\mathsf{carry}'_1 := \mathsf{carry}+\alpha_1+\alpha_2$, where α_1 and α_2 are blinding factors.

1. In the first step, P_2 generates DPF keys $(\mathsf{dpf}_0, \mathsf{dpf}_1)$ with $\alpha+\beta$ as the target and sends (dpf_0, α) to P_0 and (dpf_1, β) to P_1.
2. P_2 also computes $d_0 \leftarrow \gamma_0 \cdot \gamma_1 + \alpha_0$ and $d_1 \leftarrow \gamma_0 \cdot \gamma_1 + \alpha_1$ and sends (γ_0, d_0) to P_0 and (γ_1, d_1) to P_1 (this is used to blind the carry).
3. P_0 and P_1 then compute $\mathsf{shift}_0 \leftarrow a_{\mathsf{lo},0} - \alpha$ and $\mathsf{shift}_1 \leftarrow a_{\mathsf{lo},1} - \beta$ respectively; exchange them to get $\mathsf{shift} \leftarrow a_{\mathsf{lo},0} + a_{\mathsf{lo},1} - \alpha - \beta$.
4. P_0 and P_1 do a full-domain evaluation on dpf_0 and dpf_1 respectively and cyclically shift the output by shift. Then, P_0 and P_1 compute parities as c_0 and c_1.
5. P_0 and P_1 then exchange $(c_0+\gamma_0)$ and $(c_1+\gamma_1)$. Then they compute $\mathsf{carry}'_0 \leftarrow c_0 \cdot (c_0 + (c_1+\gamma_1)) - \gamma_0 \cdot (c_1+\gamma_1) + d_0)$ and $\mathsf{carry}'_1 \leftarrow c_1 \cdot (c_1 + (c_0+\gamma_0)) - \gamma_1 \cdot (c_0+\gamma_0) + d_1)$ respectively.

A.2 Detecting if the hi bits are 0

Let a_0 and a_1 be the shares of a. There are two possibilities (i) There is a carry-in coming in from the low bits to the hi bits, and (ii) There is no carry-in coming in.

Suppose that carry is the carry coming in from the low bits. We need to verify that $(a_{\mathsf{hi},0}+\mathsf{carry}) = (a_{\mathsf{hi},1}+\mathsf{carry})$. To compute this, P_1 generates DPF keys (with 2^l as the number of leaves) with its share of the hi bits as the target and sends the keys to P_0 and P_2 respectively. If P_0 and P_2 compute the target as $a_{\mathsf{hi},0}+\mathsf{carry}$. The parties then evaluate their DPFs at their respective targets and get x_0 and x_2 respectively. If indeed, $(a_{\mathsf{hi},0}+\mathsf{carry}) = (a_{\mathsf{hi},1}+\mathsf{carry})$, then, we'll have $x_0 \oplus x_1 = 1$, otherwise, we'll have $x_0 \oplus x_1 = 0$. Of course, this protocol as it is will not work, as this would reveal the carry out. In order to maintain the privacy, we had introduced blinding factors in the previous step and computed carry'. At the end of the protocol P_0 and P_1 end up with b_0 and b_1 with the property that, $b_0 \oplus b_1 = 1$ if a_{hi} bits are 0; otherwise $b_0 \oplus b_1 = 0$.

1. P_2 creates DPFs $(\mathsf{dpf}_0, \mathsf{dpf}_1)$ with target as (r_0+r_1) and sends (dpf_0, r_0) and (dpf_1, r_1) respectively.
2. P_0 and P_1 compute $(a_{\mathsf{hi},0}+\mathsf{carry}'_0+r_0)$ and $(a_{\mathsf{hi},1}+\mathsf{carry}'_1+r_0)$ exchange them. Then, the two parties have $\mathsf{evalat} \leftarrow (a_{\mathsf{hi},0}+a_{\mathsf{hi},1}+\mathsf{carry}'+r_0+r_1)$.
3. P_0 and P_1 then evaluate the DPF at evalat to get b_0 and b_1 respectively.

A.3 Comparing the low-bits

After computing b_0 and b_1, parties P_0 and P_1 compute, $b_0 \| a_{\mathsf{lo},0}$ and $b_1 \| a_{\mathsf{lo},1}$ and run the simple comparison protocol described in Section 7.1 on the these prepened shares. P_0 generates the DPF with the target as the low bits' share appended with the blinded c_0. Now, the target is either in the first half of the leaves or the second half of the leaves.